

We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech

Jenkins

CI/CD TOOL

Contents

- What is Jenkins?
- What is Continuous Integration?
- Continuous Integration Tools
- Jenkins Installation
- Creating Jenkins Job – GitHub Integration
- Creating Jenkins Job – Local Project
- Creating Jenkins Job – FreeStyle Project

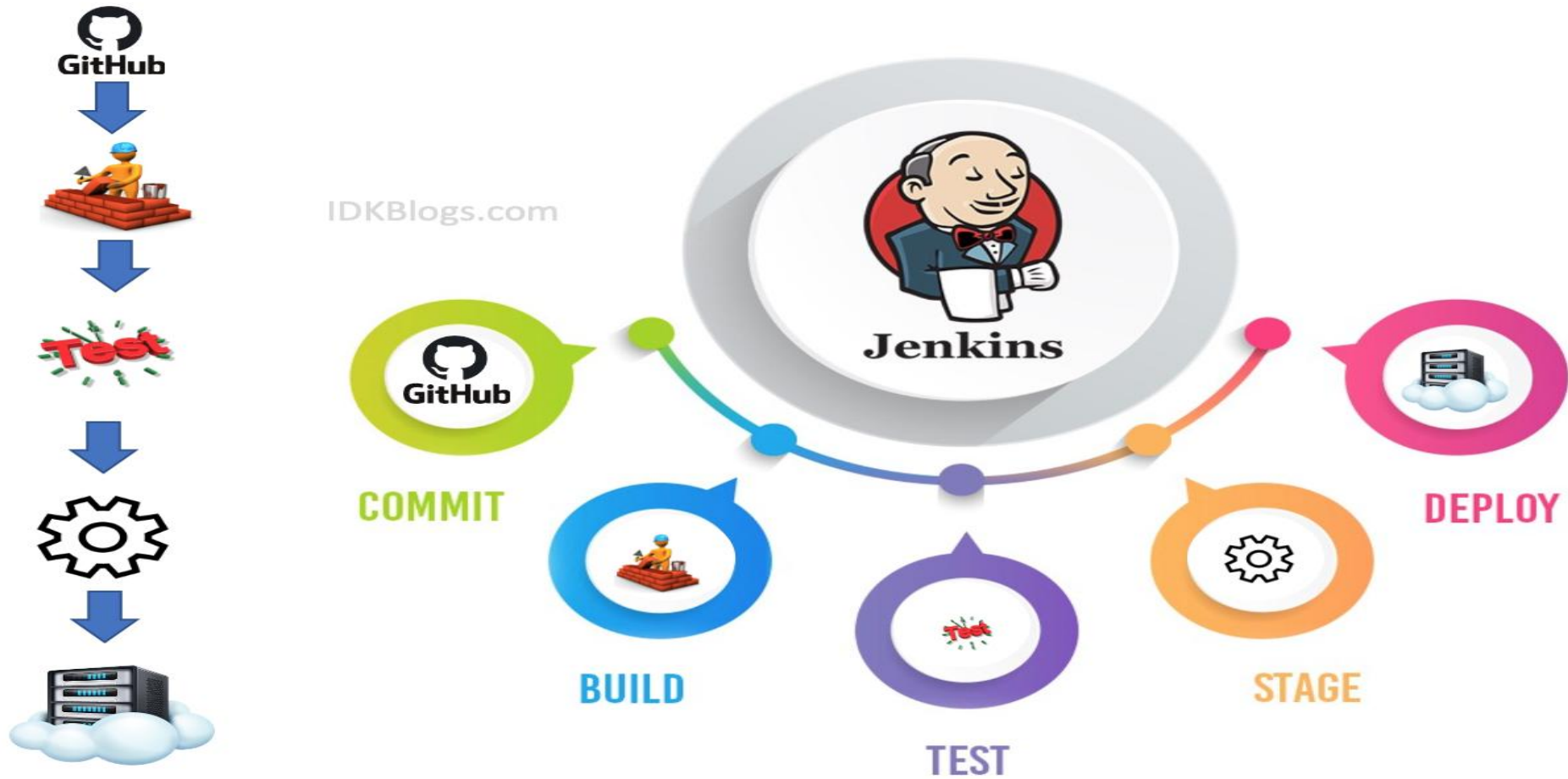
Introduction to Jenkins

What is Jenkins?

- Jenkins is an open-source automation tool written in Java with plugins built for continuous integration.
- Jenkins is used to build and test your software projects continuously making it easier for developers to integrate changes to the project, and making it easier for users to obtain a fresh build.
- It also allows you to continuously deliver your software by integrating with a large number of testing and deployment technologies.

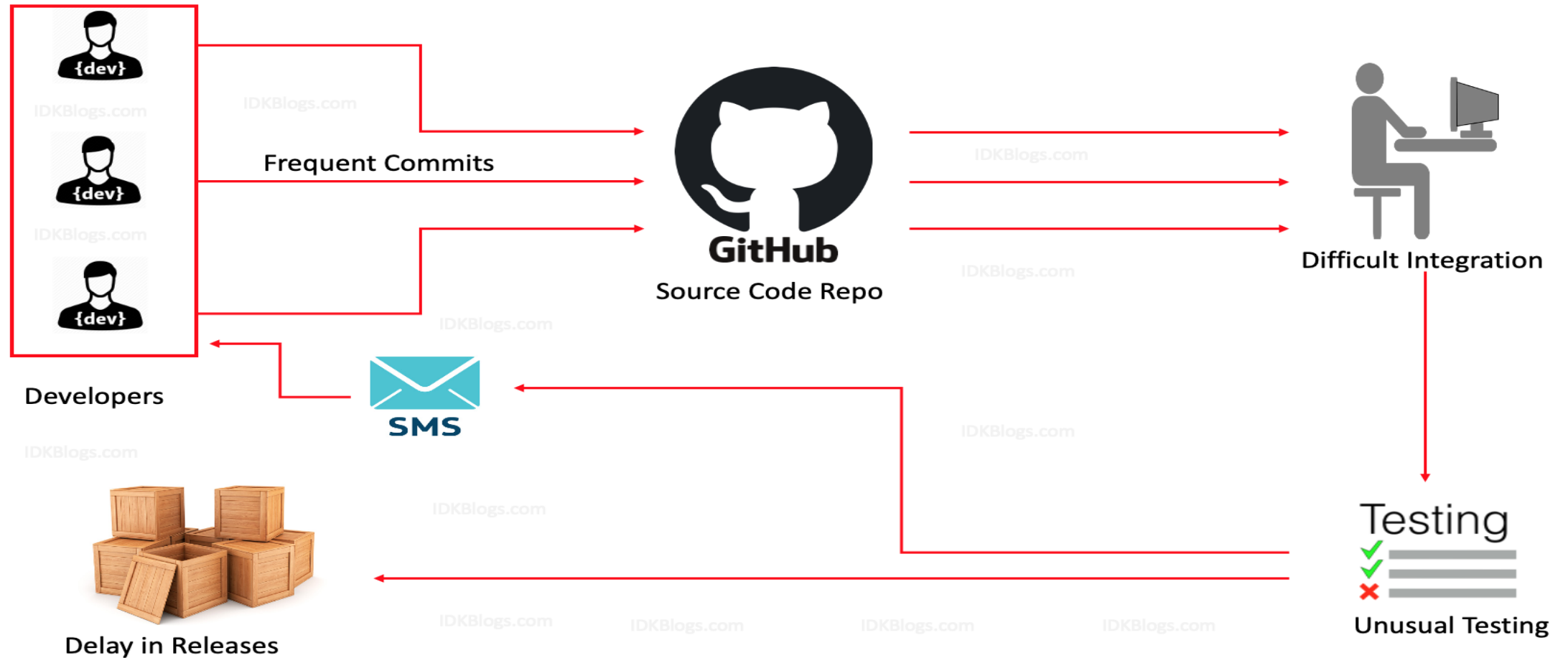
Introduction to Jenkins

What is Jenkins?



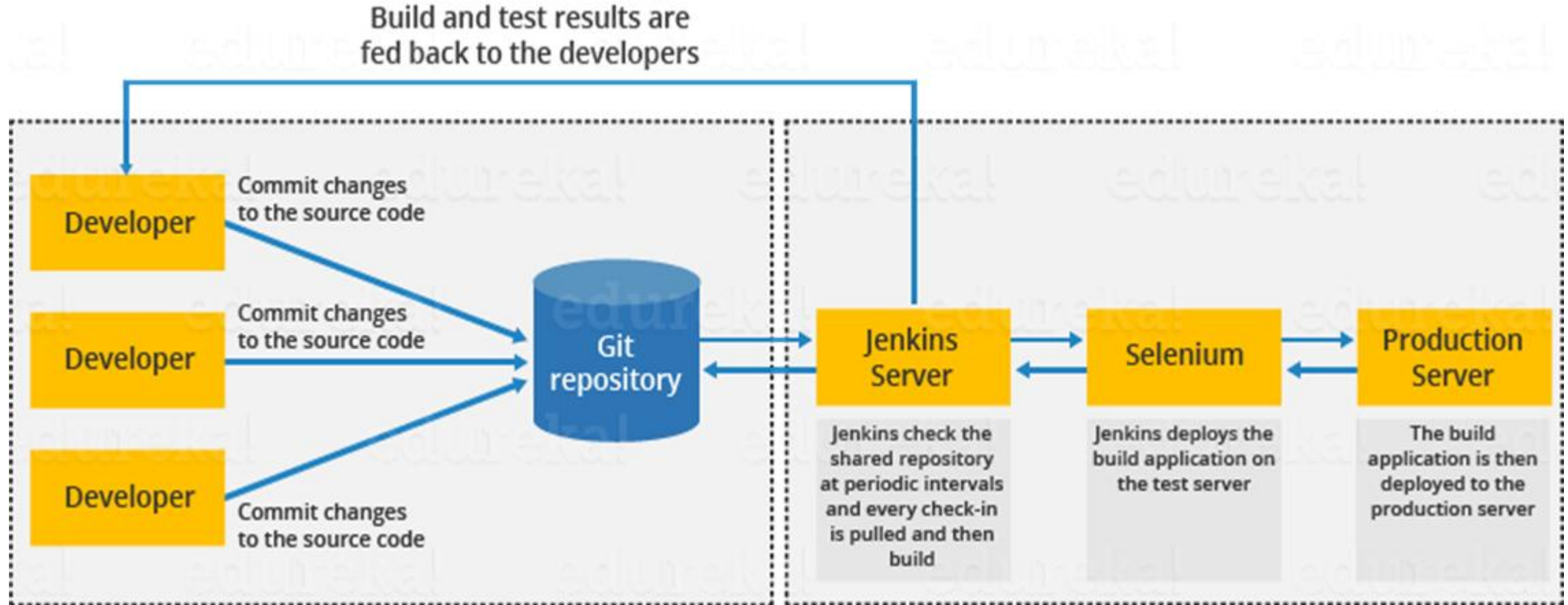
Introduction to Jenkins

What is Jenkins?



Introduction to Jenkins

What is Jenkins?



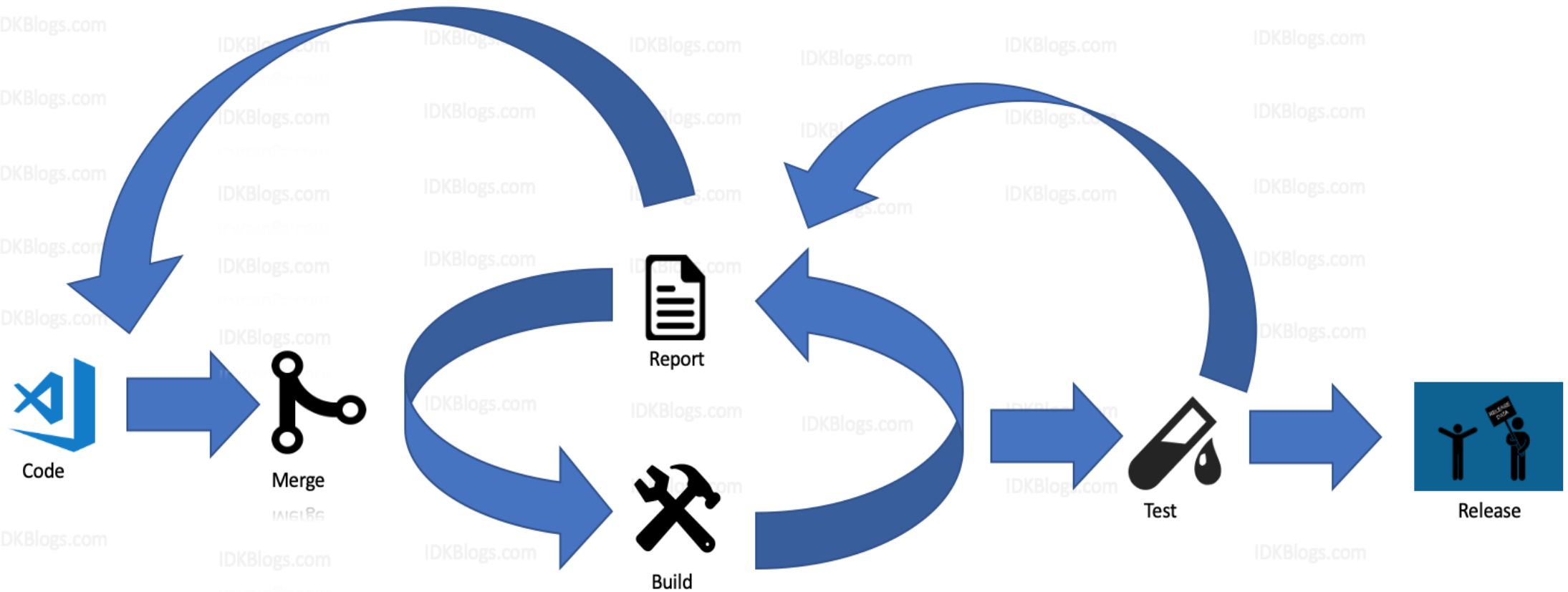
Continuous Integration

What is Continuous Integration

- Continuous Integration means Integration without breaking.
- It is a development practice in which the associated developers are doing the commits of their code changes in a shared repository many times in a day.
- When the commit and push happened in the associated branch then build is triggered.
- This build allows the developers and another associated person to detect the problems in codes early.
- In the Continuous Integration, there is so many other tasks done by CI/CD like deployment to the test server, Providing the test results, etc.

Continuous Integration

What is Continuous Integration



Continuous Integration Tools

CI Tools Comparison

1. Jenkins

- Summary: Jenkins is a veteran CI Tool with a long proven track record. It is open source and driven by community updates. Jenkins is primarily intended to an on-premise installation. Jenkins is a great option when your organization needs on-prem support for handling sensitive customer like HIPAA compliance data.

Features:

- On-premise
- Open source
- Robust addon / plugin ecosystem

Continuous Integration Tools

CI Tools Comparison

2. Bitbucket Pipelines

- Summary: Bitbucket Pipelines is a CI tool directly integrated into Bitbucket, a cloud version control system offered by Atlassian. Bitbucket Pipelines is an easy next step to enable CI if your project is already on Bitbucket. Bitbucket Pipelines are managed as code so you can easily commit pipeline definitions and kick off builds. Bitbucket Pipelines, additionally offers CD. This means projects built with Bitbucket Pipelines can be deployed to production infrastructure as well

Features:

- Easy setup and configuration
- Unified Bitbucket experience
- Cloud by 3rd party

Continuous Integration Tools

CI Tools Comparison

3. AWS CodePipeline

- Summary: Amazon Web Services is one of the most dominant cloud infrastructure providers in the market. They offer tools and services for all manner of infrastructure and code development tasks. CodePipeline is their CI Tool offering. CodePipeline can directly interface with other existing AWS tools to provide a seamless AWS experience.

Features:

- Fully cloud
- Integrated with Amazon Web services
- Custom plugin support
- Robust access control

Continuous Integration Tools

CI Tools Comparison

4. Azure Pipelines

- Summary: Azure is Microsoft's cloud infrastructure platform, the Microsoft equivalent to Amazon Web Services. Like the aforementioned AWS CodePipeline, Azure offers a CI Tool that fully integrated into the Azure suite of hosting tools.

Features:

- Azure platform integration
- Windows platform support
- Container support
- Github integration

Continuous Integration Tools

CI Tools Comparison



Jenkins Installation

Jenkins Installation

Pre-requisites:

- A system running Windows 10
- The latest copy of Java Development Kit or Java Runtime Environment installed
- Access to an account with administrator privileges

Jenkins Installation

Jenkins Installation

1. Browse to the official Jenkins download page.

- Under the Downloading Jenkins section is a list of installers for the long-term support (LTS) version of Jenkins.
- Click the Windows link to begin the download.

Jenkins Installation

Jenkins Installation

Downloading Jenkins

Jenkins is distributed as WAR files, native packages, installers, and Docker images. Follow these installation steps:

1. Before downloading, please take a moment to review the **Hardware and Software requirements** section of the User Handbook.
2. Select one of the packages below and follow the download instructions.
3. Once a Jenkins package has been downloaded, proceed to the **Installing Jenkins** section of the User Handbook.
4. You may also want to verify the package you downloaded. [Learn more about verifying Jenkins downloads.](#)

Download Jenkins 2.303.3 LTS for:

Generic Java package (.war) SHA-256: 8a6ae7367755b3f31a050faa945f7a3991abdb43d941c7294cac890c1e27
Docker
Ubuntu/Debian
CentOS/Fedora/Red Hat
Windows
openSUSE
FreeBSD 

Download Jenkins 2.319 for:

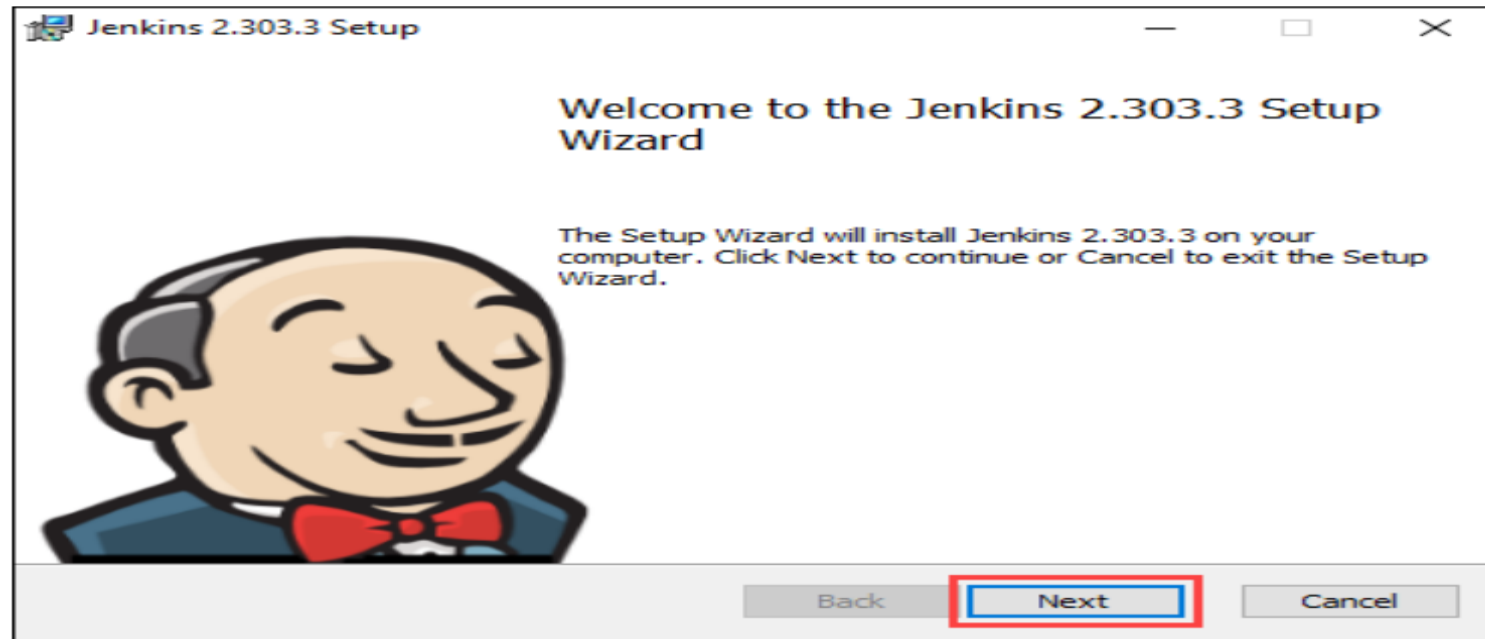
Generic Java package (.war) SHA-256: 50e9c818cda1bdf3ba7e2a1e590f027a889bd527d5bcfc2daea944ce351c7105
Docker
Ubuntu/Debian
CentOS/Fedora/Red Hat
Windows
openSUSE
Arch Linux 

of

Jenkins Installation

Jenkins Installation

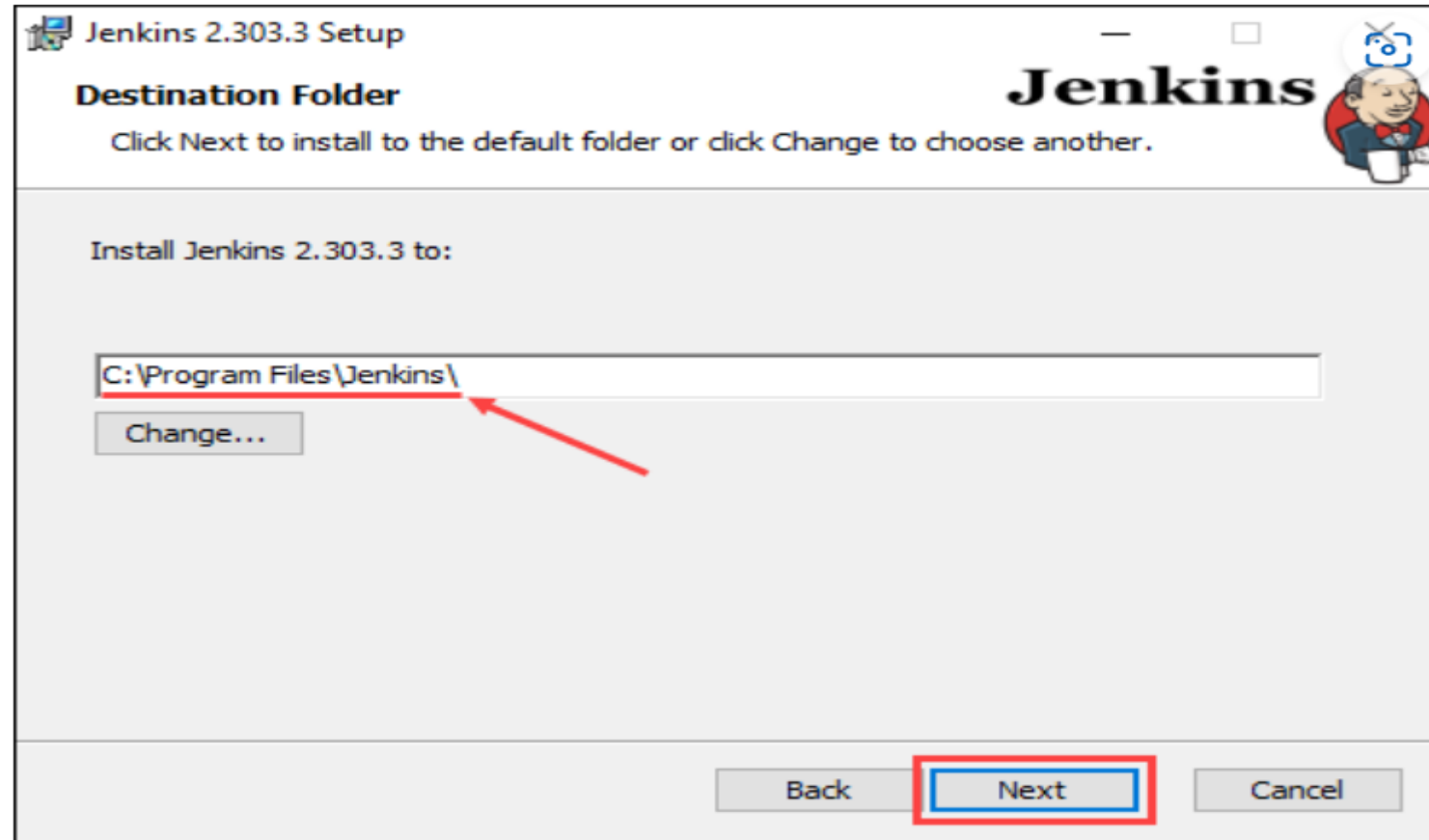
2. Once the download is complete, run the jenkins.msi installation file.
3. The setup wizard starts. Click Next to proceed.



Jenkins Installation

Jenkins Installation

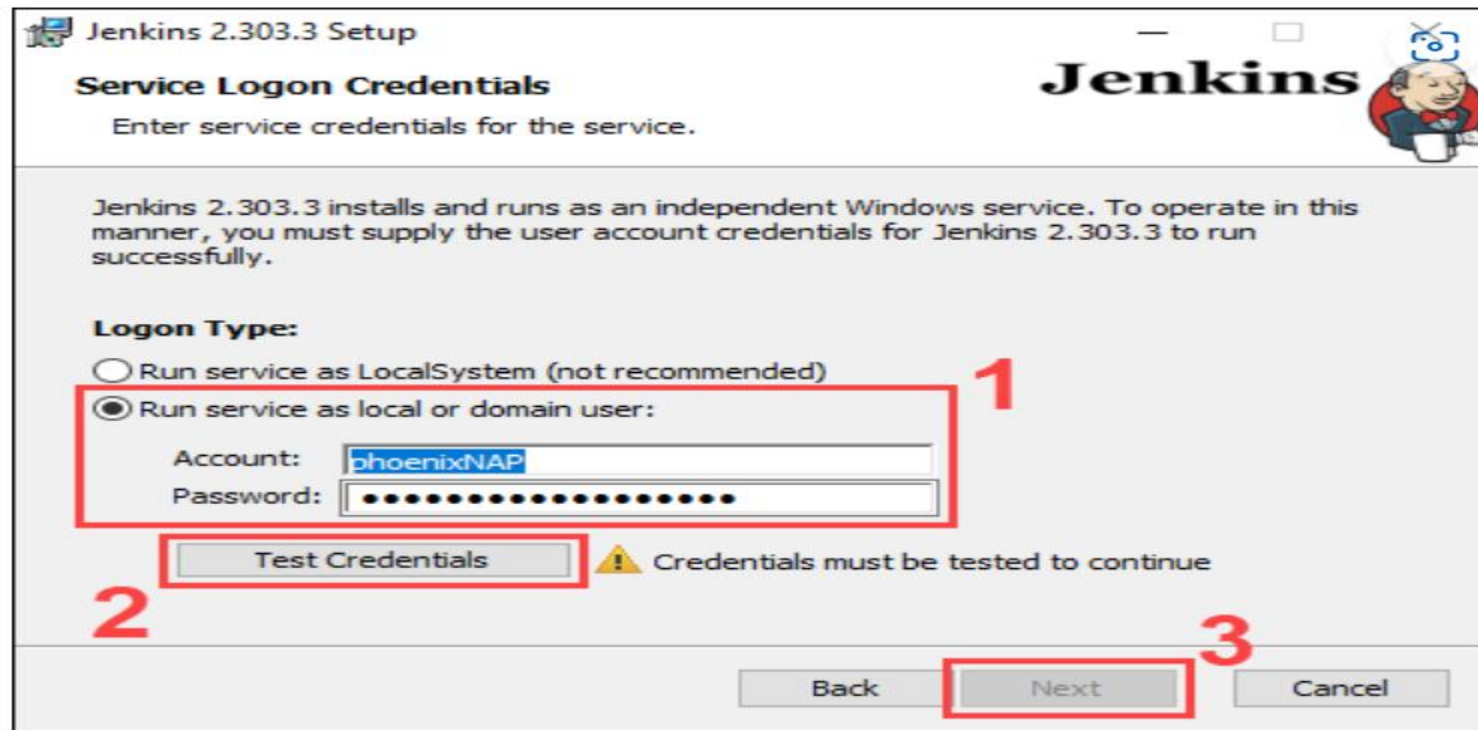
4. Select the install destination folder and click Next to continue.



Jenkins Installation

Jenkins Installation

5. Under the Run service as a local or domain user option, enter the domain username and password for the user account you want to run Jenkins with. Click Test Credentials to verify the login data, then click Next to proceed..



Jenkins 2.303.3 Setup

Service Logon Credentials
Enter service credentials for the service.

Jenkins 2.303.3 installs and runs as an independent Windows service. To operate in this manner, you must supply the user account credentials for Jenkins 2.303.3 to run successfully.

Logon Type:

☐ Run service as LocalSystem (not recommended)

☒ Run service as local or domain user:

Account:

Password:

1

2  Credentials must be tested to continue

3

Jenkins Installation

Jenkins Installation



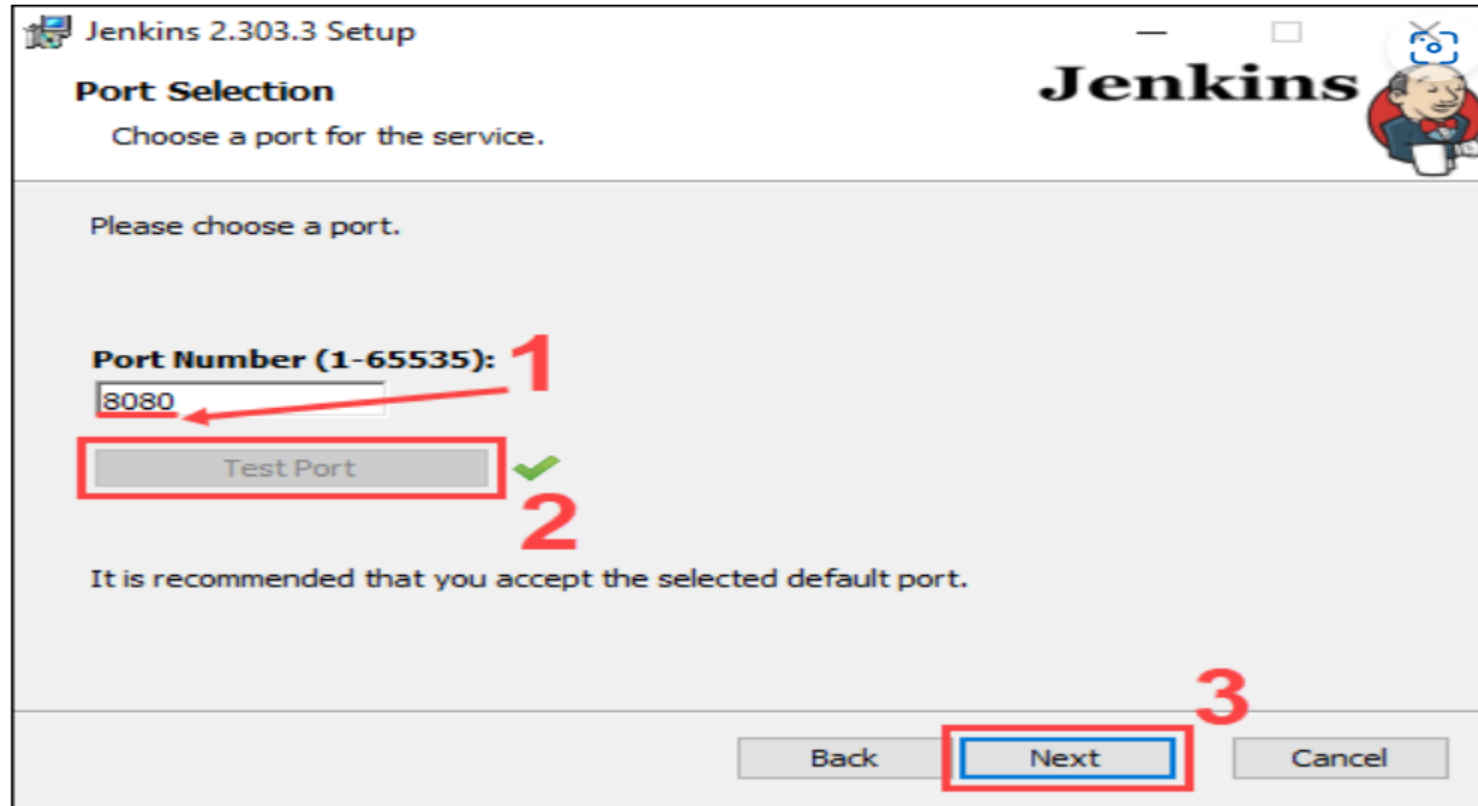
Note: When selecting the **Run service as local or domain user**, the user in question must have the required permission to log on as a service. If this is not the case, an error message stating the account cannot be verified appears. In the event of this error, perform the following steps to resolve the issue:

1. Make sure you are logged in as a user with administrative privileges.
2. In the **Administrative Tools**, open **Local Security Policy**.
3. In the **Local Security Policy** window, expand **Local Policy** in the left-hand panel and select **User Rights Assignment**.
4. In the right-hand side panel, right-click **Log on as service** and select **Properties**.
5. Click the **Add User or Group...** button.
6. Use the **Select Users or Groups** dialogue to add the current user and click **OK**.
7. Save changes by clicking the **OK** button in the **Properties** window.

Jenkins Installation

Jenkins Installation

6. Enter the port number you want Jenkins to run on. Click Test Port to check if the selected port is available, then click Next to continue.



The image shows the 'Jenkins 2.303.3 Setup' window, specifically the 'Port Selection' step. The window title is 'Jenkins 2.303.3 Setup' and the Jenkins logo is in the top right. The main text says 'Port Selection' and 'Choose a port for the service.' Below this, it says 'Please choose a port.' There is a text input field labeled 'Port Number (1-65535):' with the value '8080' entered. A red arrow points to this field with a red number '1'. Below the input field is a 'Test Port' button, which is highlighted with a red rectangle and a green checkmark, with a red number '2' next to it. At the bottom of the window, there are three buttons: 'Back', 'Next', and 'Cancel'. The 'Next' button is highlighted with a red rectangle and a red number '3'.

Jenkins 2.303.3 Setup

Port Selection
Choose a port for the service.

Please choose a port.

Port Number (1-65535): 8080

Test Port

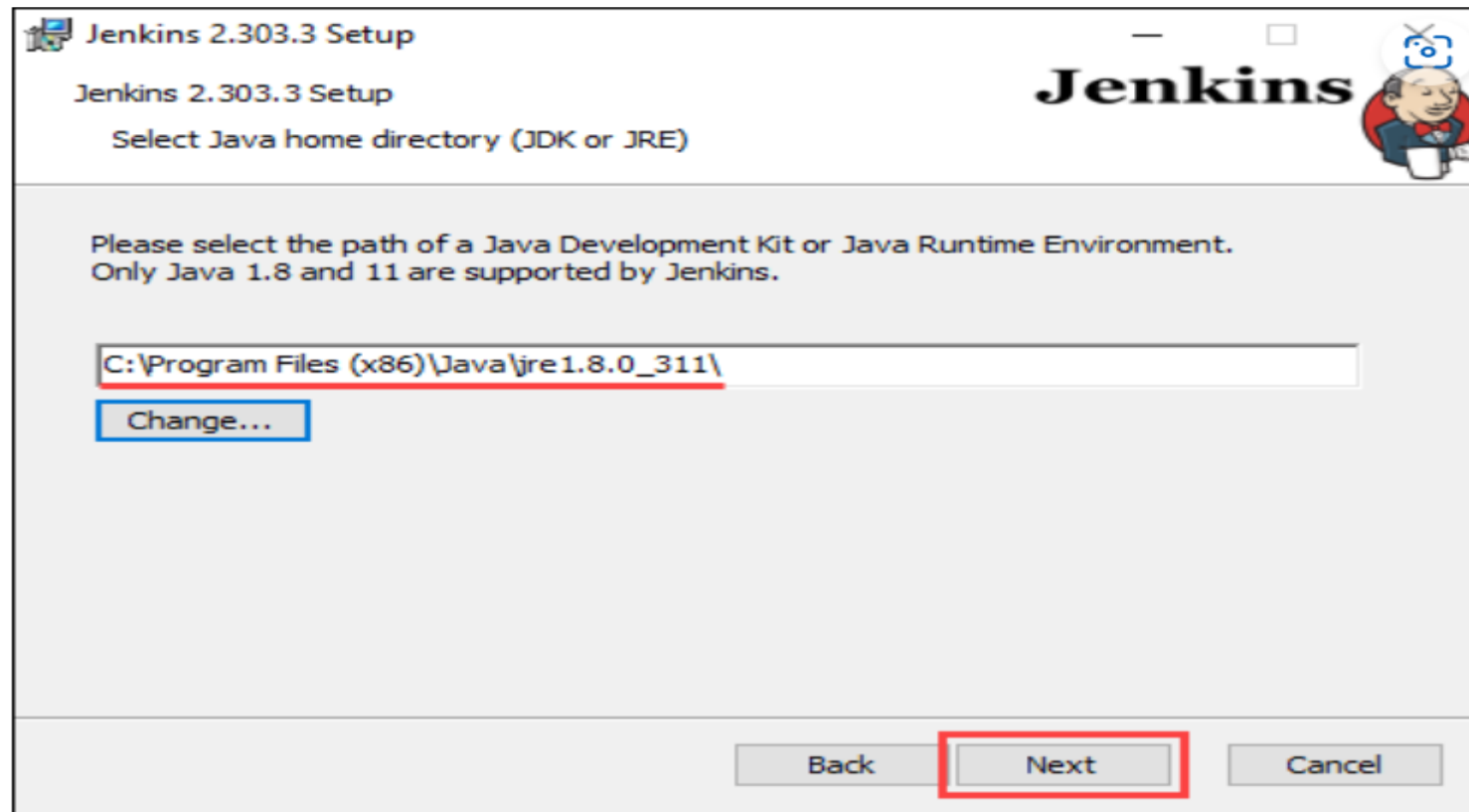
It is recommended that you accept the selected default port.

Back Next Cancel

Jenkins Installation

Jenkins Installation

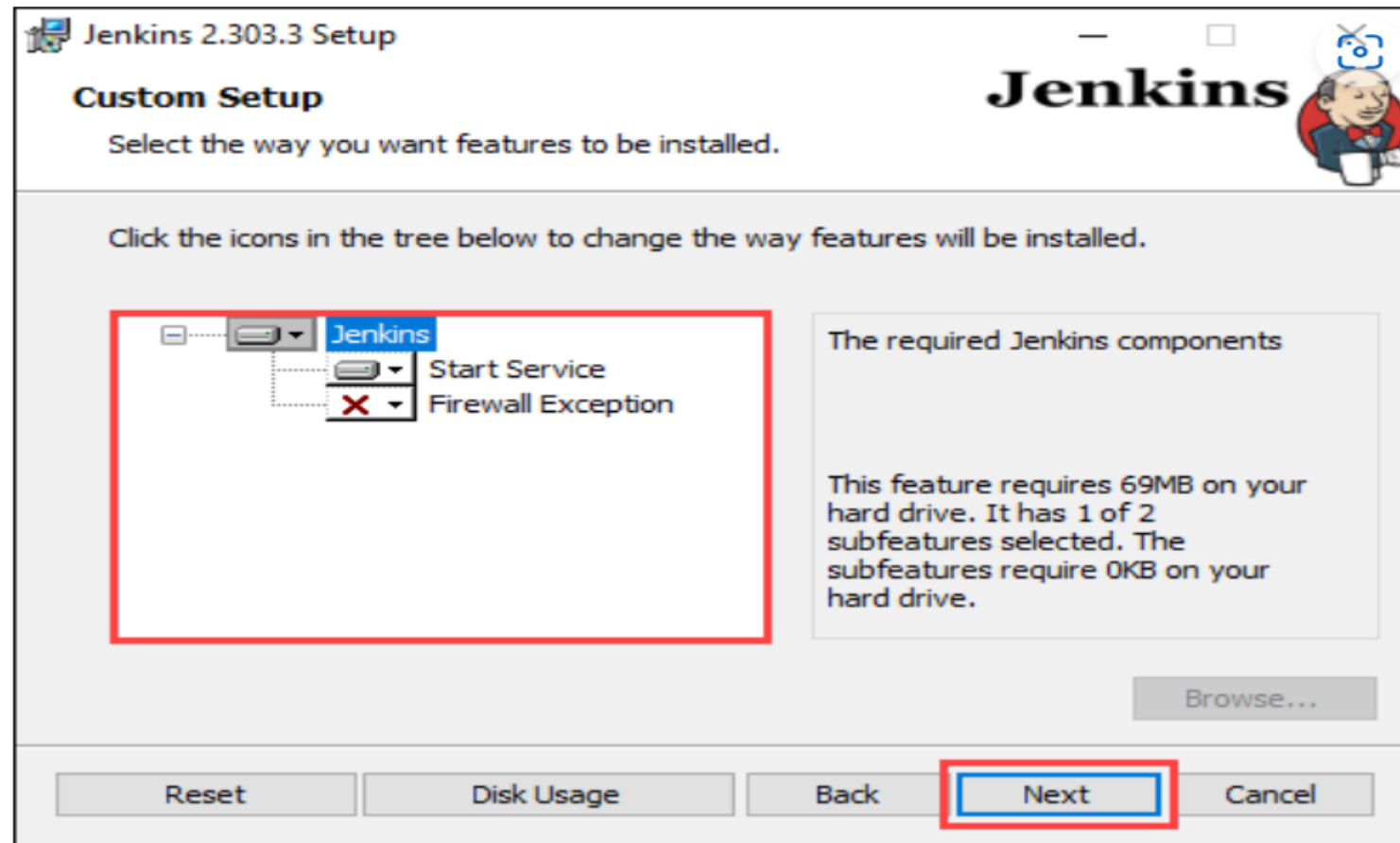
7. Select the directory where Java is installed on your system and click Next to proceed.



Jenkins Installation

Jenkins Installation

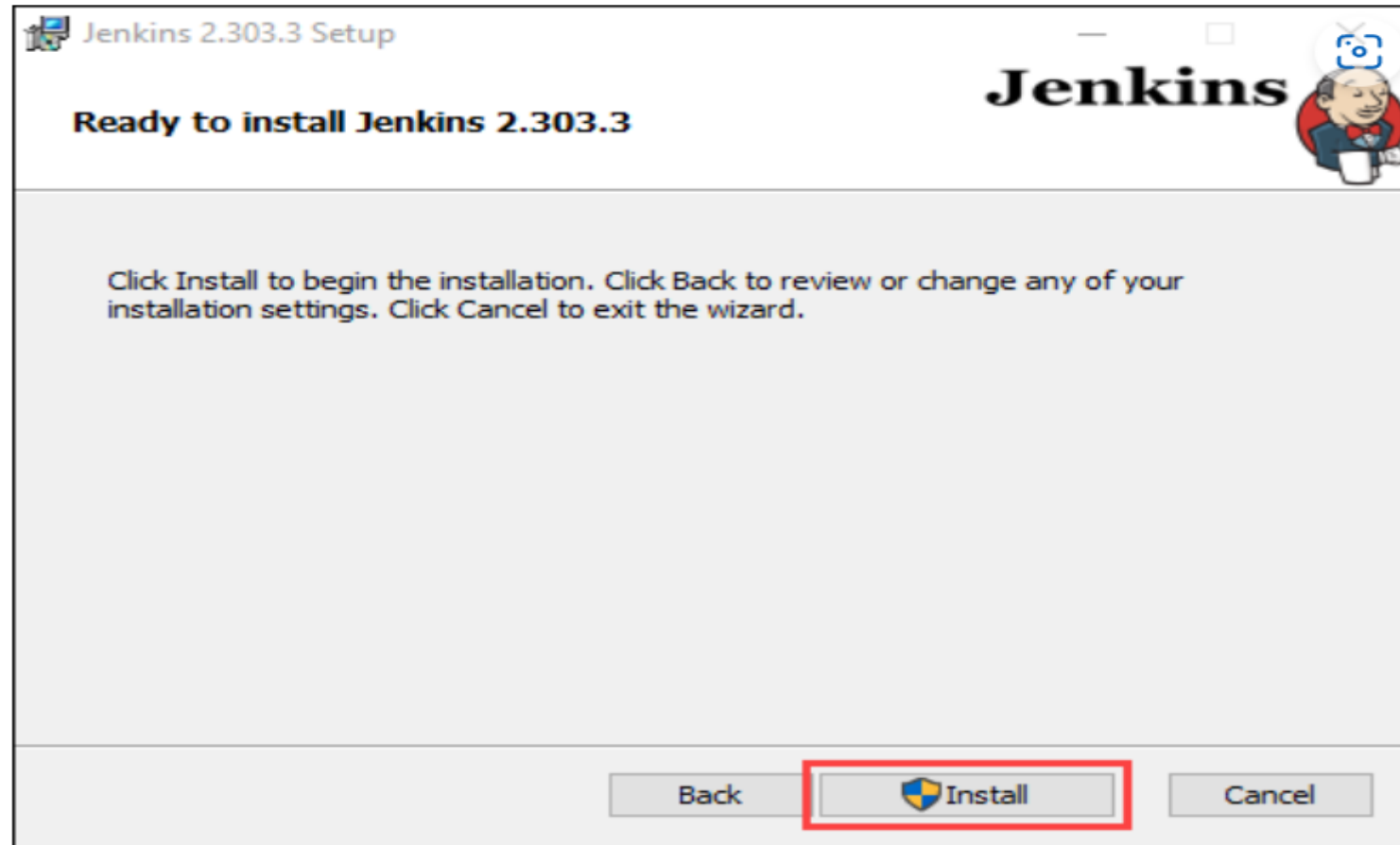
8. Select the features you want to install with Jenkins and click Next to continue.



Jenkins Installation

Jenkins Installation

9. Click Install to start the installation process.



Jenkins Installation

Jenkins Installation

10. Once the installation is complete, click Finish to exit the install wizard.



Jenkins Installation

Jenkins Installation

Unblock Jenkins

- After completing the installation process, you have to unblock Jenkins before you can customize and start using it.
1. In your web browser, navigate to the port number you selected during the installation using the following address:

```
http://localhost:[port number]
```



Jenkins Installation

Jenkins Installation

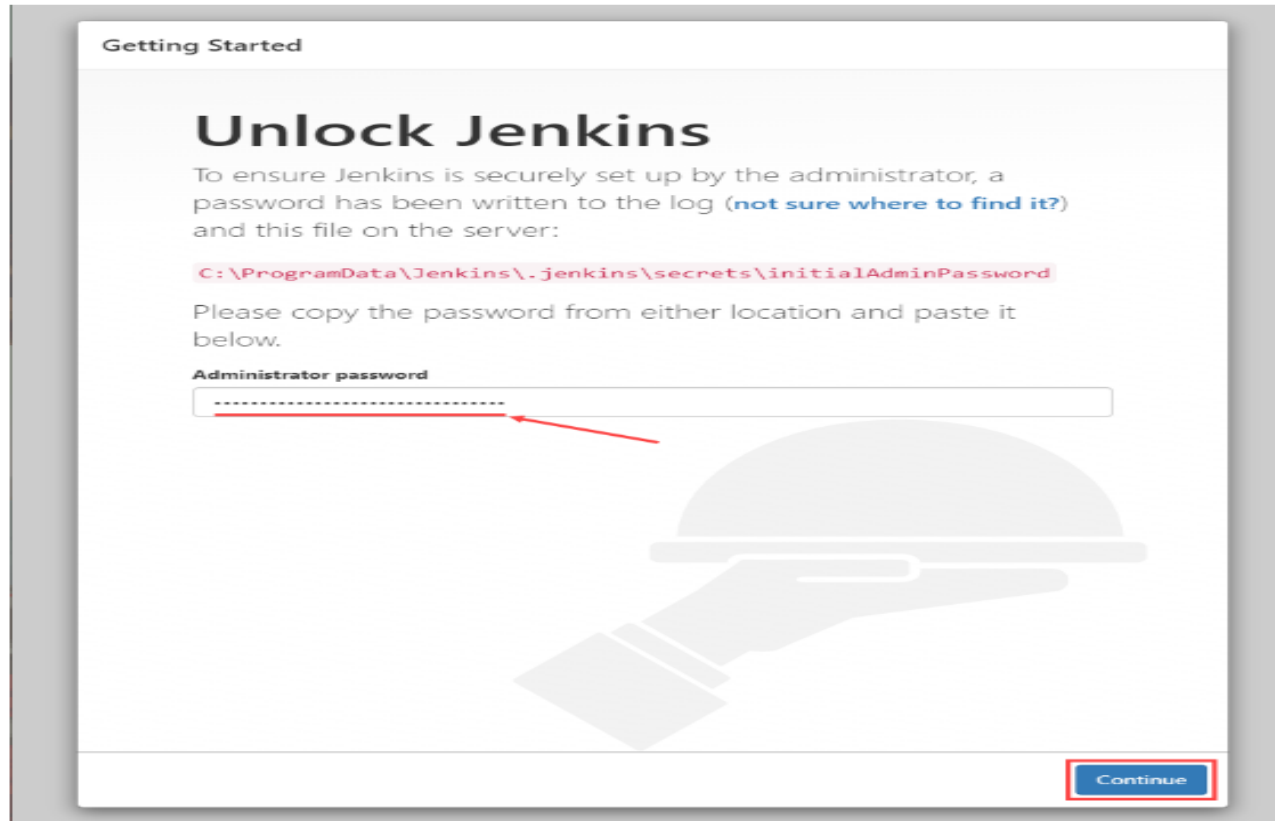
2. Navigate to the location on your system specified by the Unblock Jenkins page.
3. Open the initialAdminPassword file using a text editor such as Notepad.
4. Copy the password from the initialAdminPassword file.



Jenkins Installation

Jenkins Installation

5. Paste the password in the Administrator password field on the Unblock Jenkins page and click Continue to proceed.



The image shows the 'Unblock Jenkins' screen from the Jenkins installation process. The page has a light gray background with a white content area. At the top, it says 'Getting Started'. The main heading is 'Unblock Jenkins'. Below this, a paragraph explains that a password was written to a log file and a file on the server. The file path is shown in a red box: `C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword`. A red arrow points from this path to the 'Administrator password' input field, which is a text box with a red underline. The input field is currently empty. At the bottom right, there is a blue 'Continue' button with a red border. A large, faint watermark of a hand holding a shield is visible in the background.

Jenkins Installation

Jenkins Installation

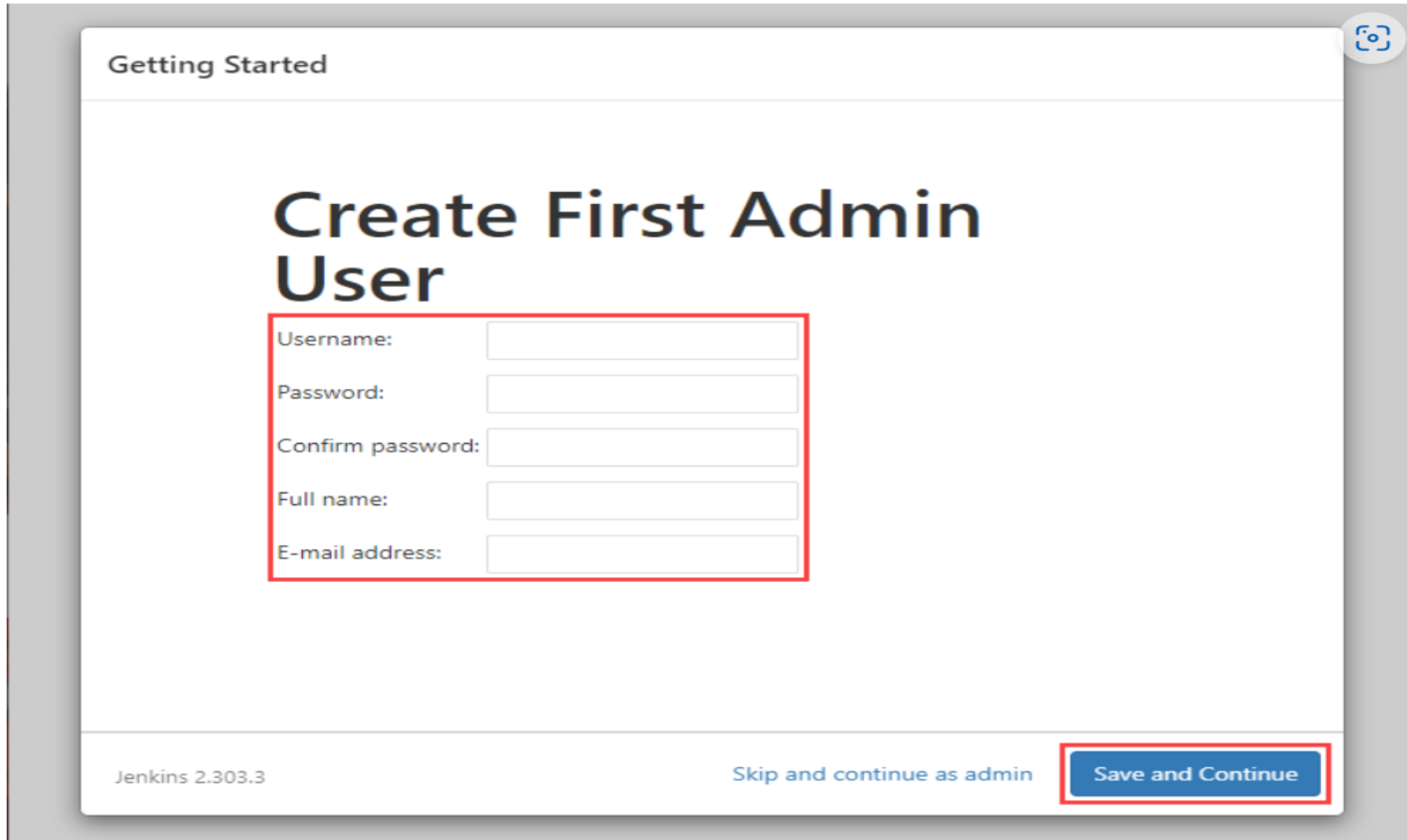
Customize Jenkins

Once you unlock Jenkins, customize and prepare the Jenkins environment.

1. Click the Install suggested plugins button to have Jenkins automatically install the most frequently used plugins.
2. After Jenkins finishes installing the plugins, enter the required information on the Create First Admin User page. Click Save and Continue to proceed.

Jenkins Installation

Jenkins Installation



The image shows the Jenkins 2.303.3 'Getting Started' screen. The main heading is 'Create First Admin User'. Below this, there is a form with five input fields: 'Username:', 'Password:', 'Confirm password:', 'Full name:', and 'E-mail address:'. The entire form is enclosed in a red rectangular border. At the bottom of the screen, there are two buttons: 'Skip and continue as admin' and 'Save and Continue'. The 'Save and Continue' button is highlighted with a red rectangular border. The Jenkins version 'Jenkins 2.303.3' is displayed in the bottom left corner.

Getting Started

Create First Admin User

Username:

Password:

Confirm password:

Full name:

E-mail address:

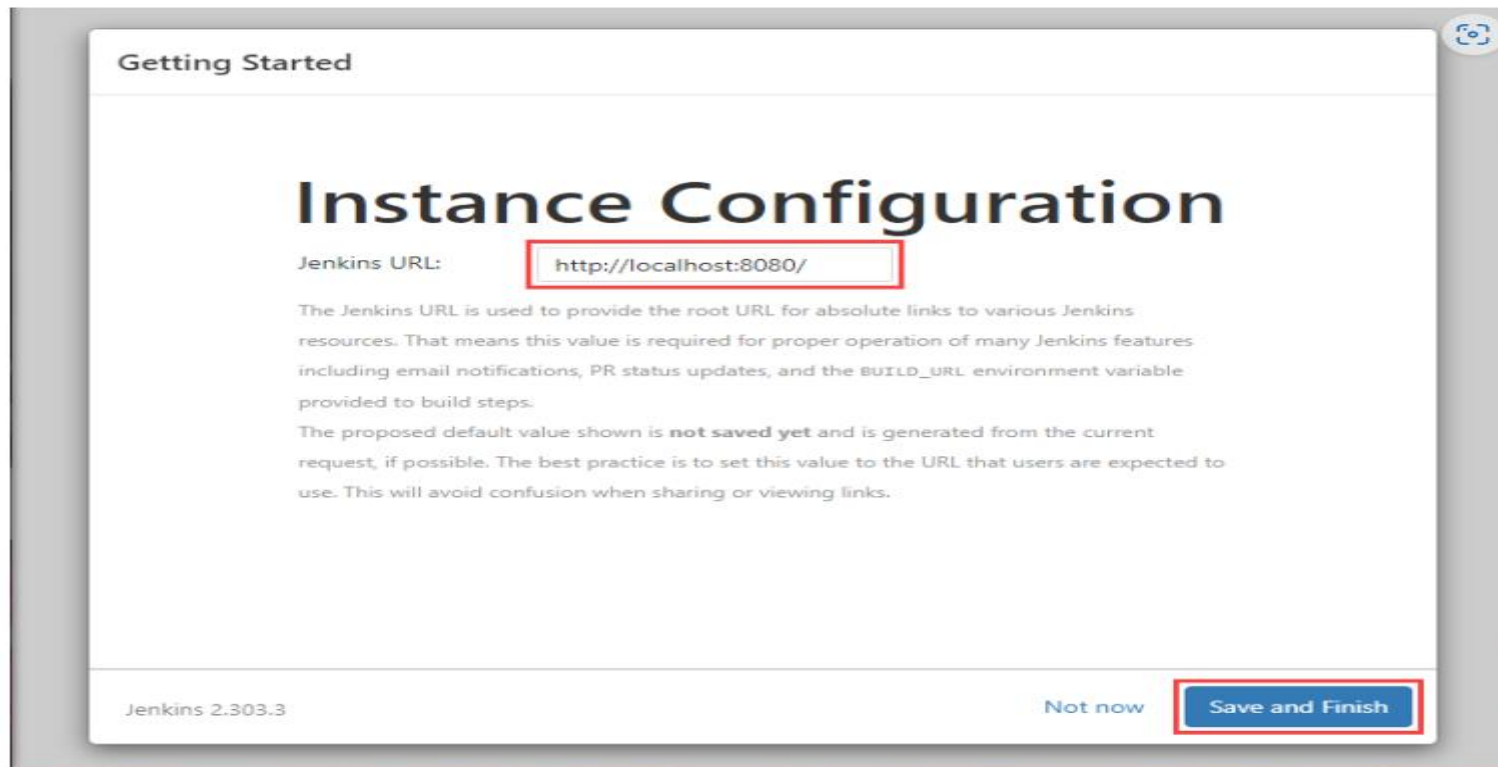
Jenkins 2.303.3

[Skip and continue as admin](#) [Save and Continue](#)

Jenkins Installation

Jenkins Installation

3. On the Instance Configuration page, confirm the port number you want Jenkins to use and click Save and Finish to finish the initial customization.



The screenshot shows the 'Getting Started' section of the Jenkins Instance Configuration page. The title 'Instance Configuration' is prominently displayed. Below it, the 'Jenkins URL' field is highlighted with a red box and contains the text 'http://localhost:8080/'. A detailed explanation follows, stating that the Jenkins URL is used for absolute links to various features like email notifications and build steps. It notes that the default value is 'not saved yet' and is generated from the current request. At the bottom right, the 'Save and Finish' button is highlighted with a red box, while the 'Not now' button is not. The Jenkins version '2.303.3' is visible in the bottom left corner.

Getting Started

Instance Configuration

Jenkins URL:

The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps.

The proposed default value shown is **not saved yet** and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.

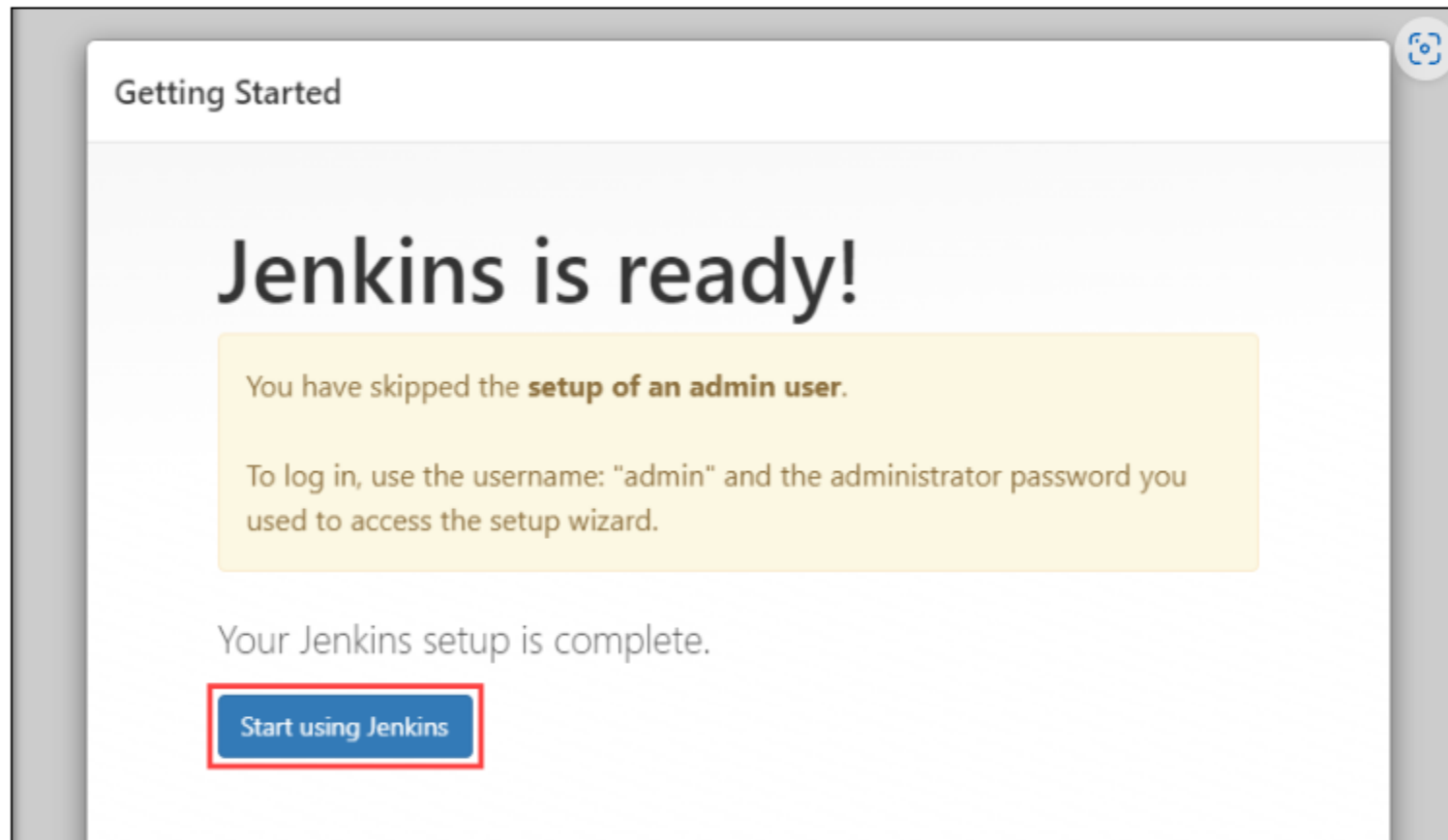
Jenkins 2.303.3

Not now

Jenkins Installation

Jenkins Installation

4. Click the Start using Jenkins button to move to the Jenkins dashboard.



Method 1 - Jenkins Job Creation – GitHub Integration



Jenkins Job Creation - GitHub

Job Creation Process – Pre-requisties and Steps

Step 1 - Configure Java and Maven in Jenkins

Step 2 – Configure GitHub in Jenkins

Step 3 – Configure Maven, HTML and TestNG reports Plugin in Jenkins

Step 4 - Create a project for running the tests using Selenium WebDriver and TestNG

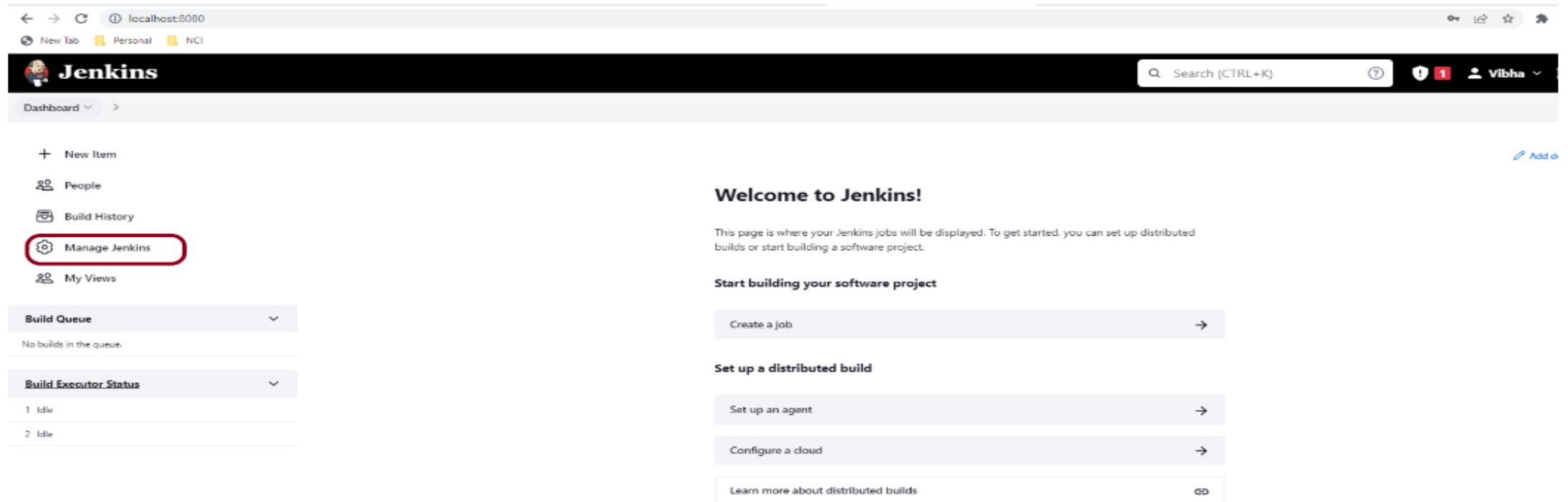
Step 5 – Create repository in GitHub and push the project

Step 6 – Jenkins GitHub Job Creation

Jenkins Job Creation - GitHub

Step 1 - Configure Java and Maven in Jenkins

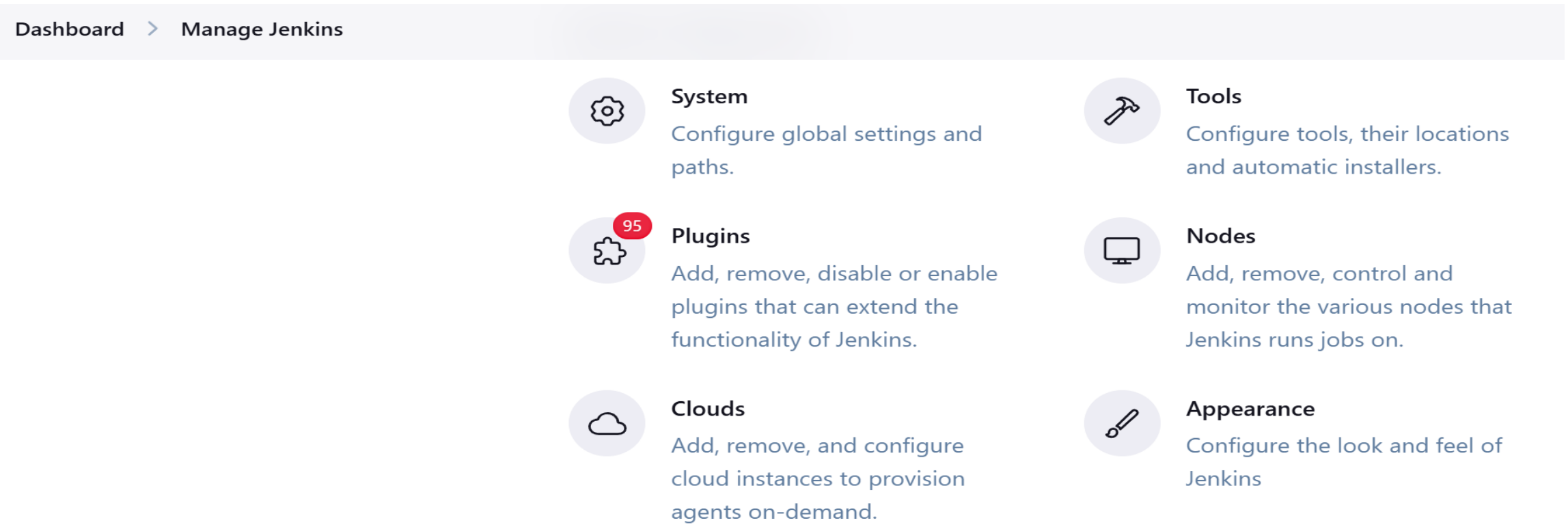
1.1 Configuring Java - Open Jenkins and go to Jenkins Dashboard. After that, click on the **Manage Jenkins** link as shown below:



Jenkins Job Creation - GitHub

Step 1 - Configure Java and Maven in Jenkins

1.1.1 When we click on the “**Manage Jenkins**” link, we are redirected to the Manage Jenkins page, where we can see various types of options, **Click “Tools” option.**



Jenkins Job Creation - GitHub

Step 1 - Configure Java and Maven in Jenkins

1.1.2 We need to set the **JDK path** in Jenkins. **Click JDK installations and Click Add JDK**

JDK installations ^  Edited

Add JDK

≡ JDK 

Name

 Required

JAVA_HOME

☐ Install automatically 

Jenkins Job Creation - GitHub

Step 1 - Configure Java and Maven in Jenkins

1.1.3 Provide the JDK's name as we gave as JDK 21 because that is what is currently installed on my machine and also provide the path of JDK in the **JAVA_HOME** textbox.

Dashboard > Manage Jenkins > Tools

JDK installations

JDK installations ^ Edited

Add JDK

≡

JDK

×

Name

JAVA_HOME

JAVA_HOME

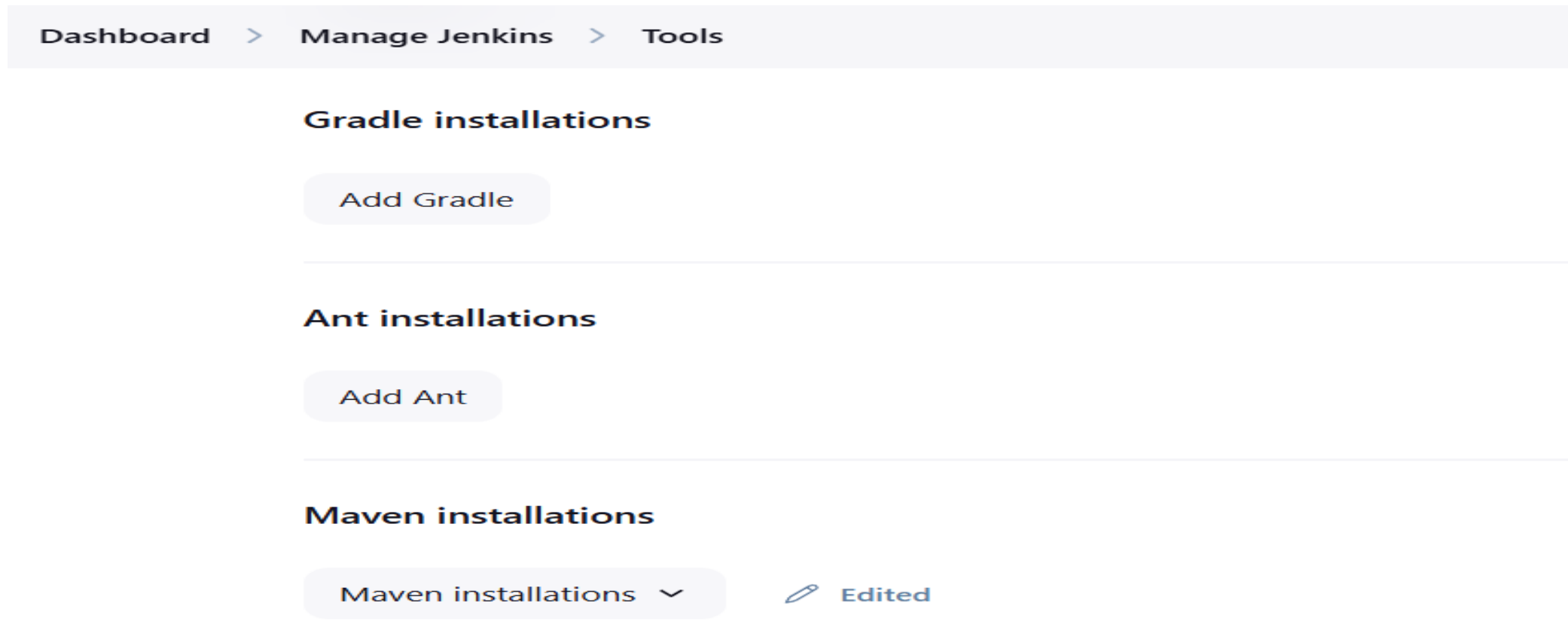
C:\Program Files\Java\jdk-21

☐ Install automatically ?

Jenkins Job Creation - GitHub

Step 1 - Configure Java and Maven in Jenkins

1.2 Configuring Maven - Click Tools option and scroll down to see the Maven option.



Jenkins Job Creation - GitHub

Step 1 - Configure Java and Maven in Jenkins

1.2.1 Click **Maven installations** and click **Add Maven**. Provide **MAVEN_HOME** in **Name Text Field** and in **MAVEN_HOME** field provide **path of Maven** installed in your machine.

Dashboard > Manage Jenkins > Tools

Maven installations ^ Edited

Add Maven

≡ **Maven**

Name

MAVEN_HOME

MAVEN_HOME

C:\Program Files\Maven

☐ Install automatically ?

Jenkins Job Creation - GitHub

Step 1 - Configure Java and Maven in Jenkins

1.3 – Click Apply and Save

Dashboard > Manage Jenkins > Tools

☰ **Maven**

Name

MAVEN_HOME

☐ Install automatically ?

Add Maven

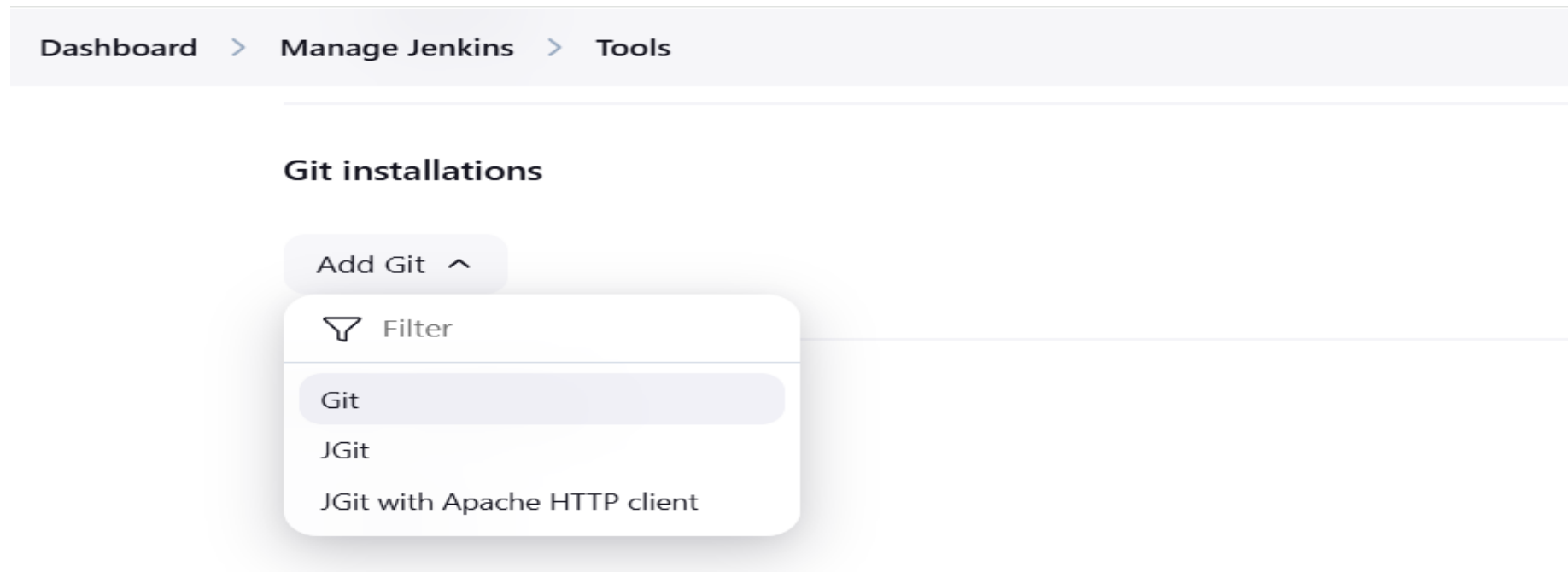
Save

Apply

Jenkins Job Creation - GitHub

Step 2 - Configure Git installations and git plugin in Jenkins

2.1.1 When we click on the “**Manage Jenkins**” link, we are redirected to the **Manage Jenkins** page, where we can see various types of options, **Click “Tools” option and got to Git installations and Click Add Git -> choose Git from dropdown**



Jenkins Job Creation - GitHub

Step 2 - Configure Git installations and git plugin in Jenkins

2.1.2 In the **Name Field** provide **Git** and in the **Path to Git executable** provide the **path of Git** installed in your machine.

Git installations

 **Git** 

Name

Path to Git executable 

☐ Install automatically 

Click Apply and Save

Jenkins Job Creation - GitHub

Step 2 - Configure Git installations and git plugin in Jenkins

2.2.1 Open your dashboard -> Click on the **Manage Jenkins** button



Jenkins Job Creation - GitHub

Step 2 - Configure Git installations and git plugin in Jenkins

2.2.2 Choose Manage Plugins.

Dashboard > Manage Jenkins

System Configuration

**System**
Configure global settings and paths.

**Tools**
Configure tools, their locations and automatic installers.

**Plugins**
Add, remove, disable or enable plugins that can extend the functionality of Jenkins.

**Nodes**
Add, remove, control and monitor the various nodes that Jenkins runs jobs on.

**Clouds**
Add, remove, and configure cloud instances to provision agents on-demand.

**Appearance**
Configure the look and feel of Jenkins

Jenkins Job Creation - GitHub

Step 2 - Configure Git installations and git plugin in Jenkins

2.2.3 Add Git Plugin

On the Plugins Page, go to the **Available** option.

- Select the GIT Plugin
- Click on Install without restart. The plugin will take a few moments to finish downloading depending on your internet connection, and will be installed automatically.
- You can also select the option Download now and Install after the restart button. In which plugin is installed after the restart
- You will be shown a “No updates available” message if you already have the Git plugin installed.

Jenkins Job Creation - GitHub

Step 2 - Configure Git installations and git plugin in Jenkins

2.2.3 Add Git Plugin

- In my case, **Git Plugin is already installed**, so you can't see it in the **Available Plugin**.

Dashboard > Manage Jenkins > Plugins

Plugins

Updates

95

Available plugins

Installed plugins

Advanced settings

git plugin

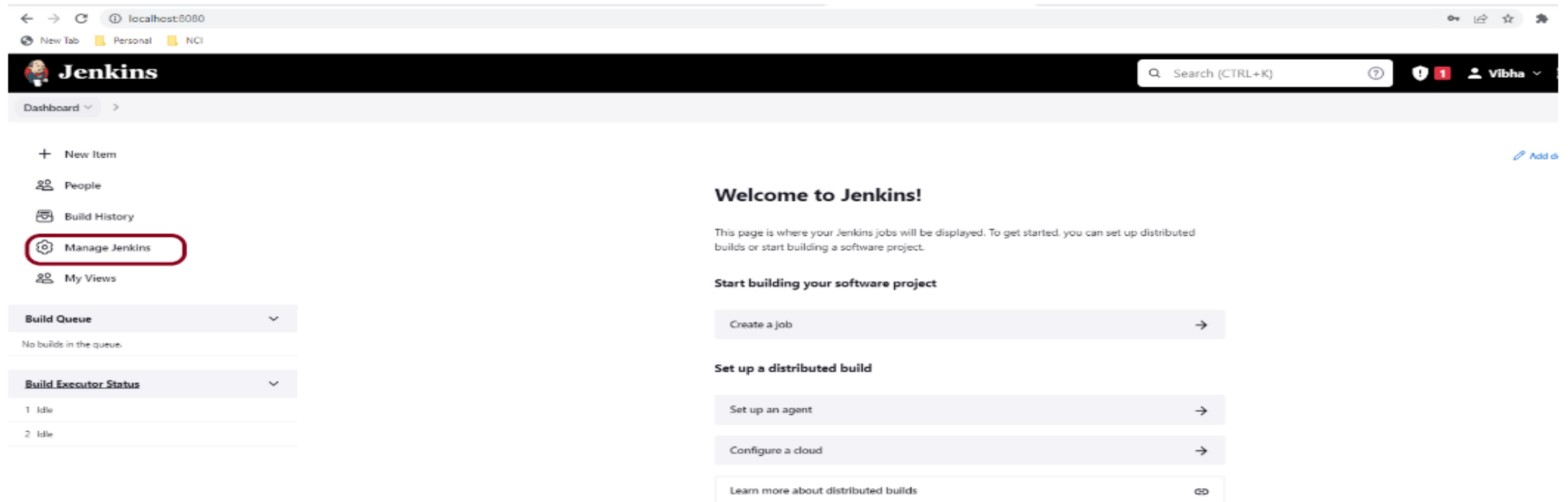
Name ↓	Enabled
Git client 6.1.1 Utility plugin for Git support in Jenkins Report an issue with this plugin Warning: The currently installed plugin version may not be safe to use. Please review the following security notices: File system information disclosure vulnerability ☒	☒
Git plugin 5.7.0 This plugin integrates Git with Jenkins. Report an issue with this plugin	☒

48 Jenkins | © SmartCliff | Internal | Version 1.0

Jenkins Job Creation - GitHub

Step 3 – Configure Maven, HTML and TestNG reports Plugin in Jenkins

1.1 Configuring Java - Open Jenkins and go to Jenkins Dashboard. After that, click on the **Manage Jenkins** link as shown below:



Jenkins Job Creation - GitHub

Step 3.1: Manage Jenkins ->Plugins

Dashboard >

new item

People

Build History

Project Relationship

Check File Fingerprint

Manage Jenkins

My Views

Build Queue

No builds in the queue.

Build Executor Status

built-in node (0 of 2 executors busy)

All

+

S	W	Name ↓	Last Success	Last Failure	Last Duration
		project1	N/A	26 days #2	4 ms
		Sample1	2 days 9 hr #15	20 days #3	1 min 22 sec
		sample2	1 day 8 hr #11	N/A	55 sec
		Sample3	N/A	2 days 10 hr #1	2.1 sec
		Sample4	2 days 8 hr #3	N/A	1 min 3 sec
		Testng Selenium	N/A	2 days 10 hr #4	2.1 sec

Icon:

S

M

L

Icon legend

Atom feed for all

Atom feed for failures

Atom feed for just latest builds

50 Jenkins | © SmartCliff | Internal | Version 1.0

Jenkins Job Creation - GitHub

Step 3.2: Manage Jenkins ->Plugins

Dashboard > Manage Jenkins

System Configuration



System
 Configure global settings and paths.



Tools
 Configure tools, their locations and automatic installers.



Plugins
 Add, remove, disable or enable plugins that can extend the functionality of Jenkins.



Nodes and Clouds
 Add, remove, control and monitor the various nodes that Jenkins runs jobs on.

Security



Security
 Secure Jenkins; define who is allowed to access/use the system.



Credentials
 Configure credentials



Credential Providers
 Configure the credential providers and types

localhost:8082/manage/pluginManager

Activate Windows
 Go to Settings to activate Windows.

51 Jenkins | © SmartCliff | Internal | Version 1.0

Jenkins Job Creation - GitHub

Step 3.3: Search Maven Integration and install


Jenkins

?
🔔 1
🛡️ 2
👤 viji19904feb
🔓 log out

Dashboard > Manage Jenkins > Plugins

📄 Updates
 📁 Available plugins
 ⚙️ Installed plugins
 ⚙️ Advanced settings

Plugins

Name ↓	Enabled
Maven Integration plugin 3.23 This plugin provides a deep integration between Jenkins and Maven. It adds support for automatic triggers between projects depending on SNAPSHOTS as well as the automated configuration of various Jenkins publishers such as Junit. Report an issue with this plugin	☒ ✖

Activate Windows
Go to Settings to activate Windows.
REST API Jenkins 2.401.3

Jenkins Job Creation - GitHub

Step 3.4: Search testng results and install plugin.


Jenkins

?
🔔 1
🛡️ 2
👤 viji19904feb
🚪 log out

Dashboard > Manage Jenkins > Plugins

📄 Updates
 📁 Available plugins
 ⚙️ **Installed plugins**
⚙️ Advanced settings

Plugins

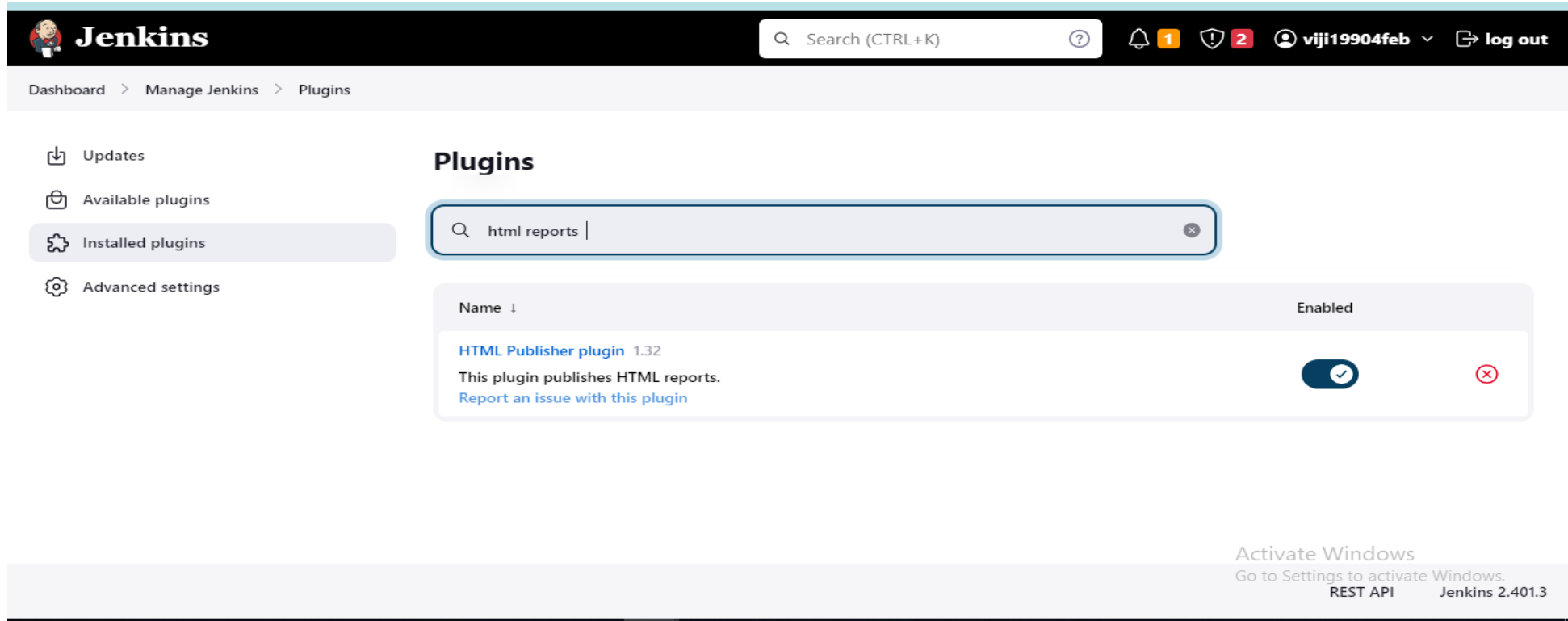
Name ↓	Enabled
TestNG Results Plugin 794.ve7e315837b_b_3 This plugin integrates TestNG test reports to Jenkins. Report an issue with this plugin	☒ ✖

Activate Windows
Go to Settings to activate Windows.

REST API Jenkins 2.401.3

Jenkins Job Creation - GitHub

Step 3.5: Search html reports and install the plugins.



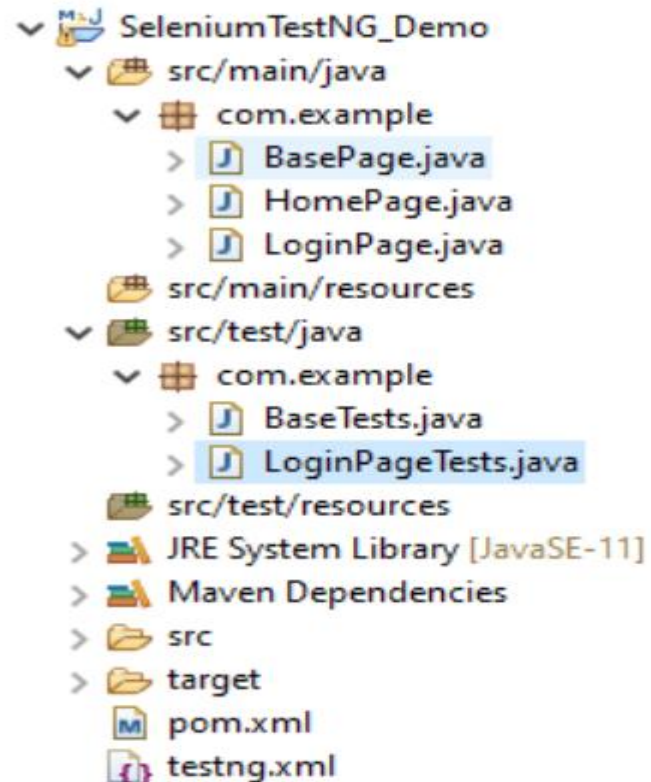
The screenshot shows the Jenkins web interface. At the top, there's a navigation bar with the Jenkins logo, a search bar, and user information. Below this is a breadcrumb trail: Dashboard > Manage Jenkins > Plugins. On the left sidebar, there are links for Updates, Available plugins, Installed plugins (which is highlighted), and Advanced settings. The main content area is titled 'Plugins' and features a search bar with the text 'html reports'. Below the search bar, a table lists installed plugins. The first entry is the 'HTML Publisher plugin' version 1.32, which is described as 'This plugin publishes HTML reports.' and has a link to 'Report an issue with this plugin'. The plugin is shown as 'Enabled' with a toggle switch and a close button. At the bottom right, there is a 'Activate Windows' watermark.

Name	Enabled
HTML Publisher plugin 1.32 This plugin publishes HTML reports. Report an issue with this plugin	☒

Jenkins Job Creation - GitHub

Step 4 – Create a project for running the tests using Selenium WebDriver and TestNG

- We need to create a running project to automate any web application using Selenium WebDriver, Page Object Model, Data Driven Testing and TestNG



Jenkins Job Creation - GitHub

Step 4 – Create a project for running the tests using Selenium WebDriver and TestNG

- Pom.xml file

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-  
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-  
4.0.0.xsd">  
  <modelVersion>4.0.0</modelVersion>  
  <groupId>com.testng</groupId>  
  <artifactId>TestNG-Jenkins</artifactId>  
  <version>0.0.1-SNAPSHOT</version>  
  <properties>  
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>  
    <selenium.version>4.30.0</selenium.version>  
    <testng.version>7.11.0</testng.version>  
    <maven.compiler.plugin.version>3.5.1</maven.compiler.plugin.version>  
    <maven.compiler.source>1.8</maven.compiler.source>  
    <maven.compiler.target>1.8</maven.compiler.target>
```

Jenkins Job Creation - GitHub

Step 4 – Create a project for running the tests using Selenium WebDriver and TestNG

- Pom.xml file

```
<maven.surefire.plugin.version>3.0.0-M5</maven.surefire.plugin.version>
</properties>
<dependencies>
<!-- https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
<dependency>
<groupId>org.seleniumhq.selenium</groupId>
<artifactId>selenium-java</artifactId>
<version>${selenium.version}</version>
</dependency>
<!-- https://mvnrepository.com/artifact/org.testng/testng -->
<dependency>
<groupId>org.testng</groupId>
<artifactId>testng</artifactId>
<version>${testng.version}</version>
<scope>test</scope>
</dependency>
```


Jenkins Job Creation - GitHub

Step 4 – Create a project for running the tests using Selenium WebDriver and TestNG

- Pom.xml file

```
</dependencies>
<build>
<plugins>
<!-- Compiler plug-in -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>${maven.compiler.plugin.version}</version>
<configuration>
<source>${maven.compiler.source}</source> <!--For JAVA 8 use 1.8-->
<target>${maven.compiler.target}</target> <!--For JAVA 8 use 1.8-->
</configuration>
</plugin>
```

Jenkins Job Creation - GitHub

Step 4 – Create a project for running the tests using Selenium WebDriver and TestNG

- Pom.xml file

```
<!-- Added Surefire Plugin configuration to execute tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>${maven.surefire.plugin.version}</version>
<configuration>
<suiteXmlFiles>
<suiteXmlFile>TestNG.xml</suiteXmlFile>
</suiteXmlFiles>
<testFailureIgnore>>false</testFailureIgnore>
<reportsDirectory>${project.build.directory}/surefire-reports</reportsDirectory>
<argLine>
-javaagent:"${settings.localRepository}/org/aspectj/aspectjweaver/${aspectj.version}/aspectjweaver-${aspectj.version}.jar"
</argLine>
</configuration>
</plugin>
</plugins>
</build>
</project>
```

***Note: testng.xml file should be created with in the same project**

Jenkins Job Creation - GitHub

Step 4 – Create a project for running the tests using Selenium WebDriver and TestNG

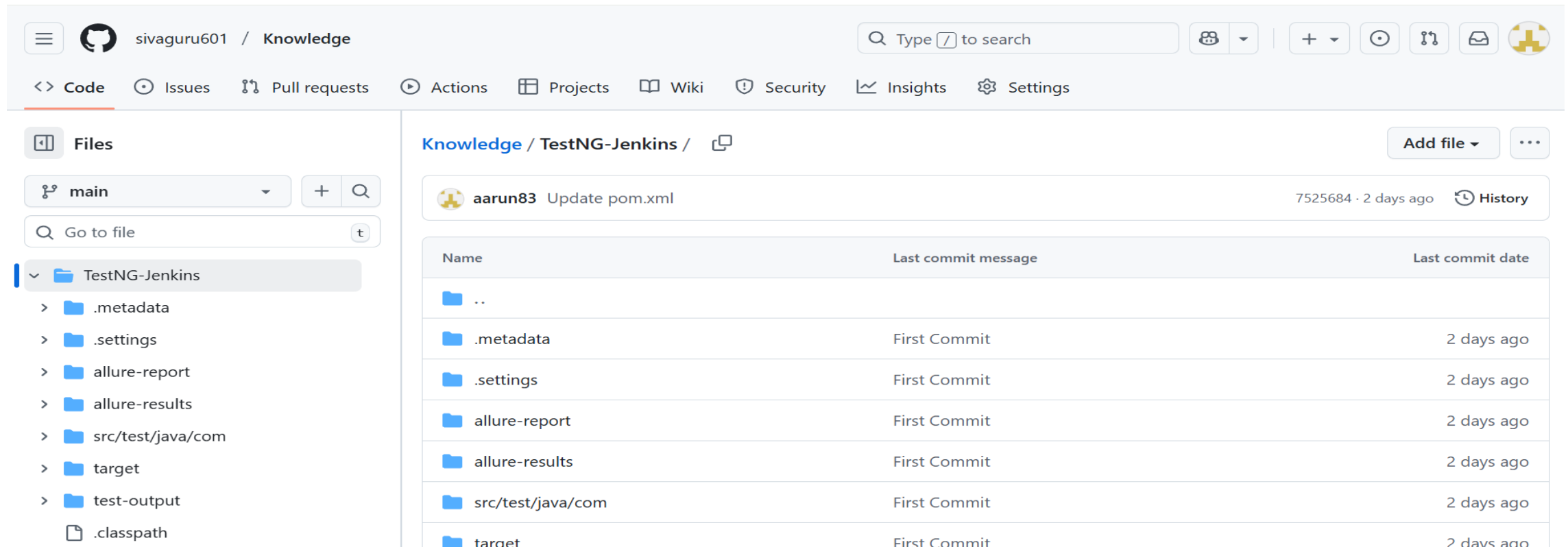
- TestNG.xml file

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name = "Allure Reports" parallel="classes" thread-count="3">
  <test name = "Login Page Tests">
    <classes>
      <class name = "com.Tests.LoginTests"/>
    </classes>
  </test>
  <test name = " Dashboard Tests">
    <classes>
      <class name = "com.Tests.DashboardTests"/>
    </classes>
  </test>
</suite>
```

Jenkins Job Creation - GitHub

Step 5 – Create repository in GitHub and push the project

- If your Maven project currently exists only on your local machine. Then
- Create a new **GitHub repository**. Push your local **Maven project** to it.



Knowledge / TestNG-Jenkins /

7525684 · 2 days ago History

Name	Last commit message	Last commit date
..		
.metadata	First Commit	2 days ago
.settings	First Commit	2 days ago
allure-report	First Commit	2 days ago
allure-results	First Commit	2 days ago
src/test/java/com	First Commit	2 days ago
target	First Commit	2 days ago

Jenkins Job Creation - GitHub

Step 6.1 : Click New Item in home page.


Jenkins

?
🔔 1
🛡️ 2
👤 viji19904feb
🚪 log out

Dashboard >

+ New Item

👤 People

📅 Build History

😊 Project Relationship

🔍 Check File Fingerprint

⚙️ Manage Jenkins

📁 My Views

All +

S	W	Name ↓	Last Success	Last Failure	Last Duration
❌	☁️	project1	N/A	23 days #2	4 ms ▶
✅	☀️	Sample1	2 min 17 sec #8	18 days #3	1 min 12 sec ▶
✅	☀️	sample2	7 days 15 hr #7	N/A	2 min 29 sec ▶
❌	☁️	Testng Selenium	N/A	23 days #3	0.2 sec ▶

Icon: S M L

Icon legend

📡 Atom feed for all

📡 Atom feed for failures

📡 Atom feed for just latest builds

Build Queue

No builds in the queue.

Build Executor Status

localhost:8082/view/all/newJob (busy)

✎ Add description

Jenkins Job Creation - GitHub

Step 6.2: Enter item name and select Maven Project and click OK


Jenkins

?
🔔 1
🛡️ 2
👤 viji19904feb
🚪 log out

Dashboard > All >

Enter an item name

» Required field


Freestyle project

This is the central feature of Jenkins. Jenkins will build your project, combining any SCM with any build system, and this can be even used for something other than software build.


Maven project

Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.


Pipeline

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.


Multi-configuration project

Activate Windows
Go to Settings to activate Windows.

Step 6.3: Source Code Management Section

- You will see a **Git** option under **Source Code Management** if your Git plugin has been installed in Jenkins. Enter the **Git repository URL** to pull the code from **GitHub**. Enter the **credentials** to log in to **GitHub**.

Dashboard > JenkinsMavenGitDemo > Configuration

Configure

- General
- Source Code Management**
- Build Triggers
- Build Environment
- Pre Steps
- Build
- Post Steps
- Build Settings
- Post-build Actions

☐ None

☒ Git ?

Repositories ?

Repository URL ?

https://github.com/sivaguru601/Knowledge.git

Credentials ?

aarun.83@gmail.com/*****

+ Add

Advanced ▾

Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/main

Step 6.3: Source Code Management Section

- I have already set up the credentials to log in to GitHub. For first-time users, **click on the Add button and select Jenkins.**

Source Code Management

☐ None

☒ Git ?

Repositories ?

Repository URL ?

https://github.com/vibssingh/Selenium-TestNG.git

Credentials ?

vibssingh/***** (GitHub Credentials)

+ Add



Jenkins

Advanced

Jenkins Credentials Provider

Step 6.3: Source Code Management Section

- Add the **username and password used to log in to GitHub**. Click on the “Add” button.

Jenkins Credentials Provider: Jenkins

Add Credentials

Domain

Global credentials (unrestricted) ▼

Kind

Username with password ▼

Scope ?

Global (Jenkins, nodes, items, all child items, etc) ▼

Username ?

vibhasingh2004@gmail.com

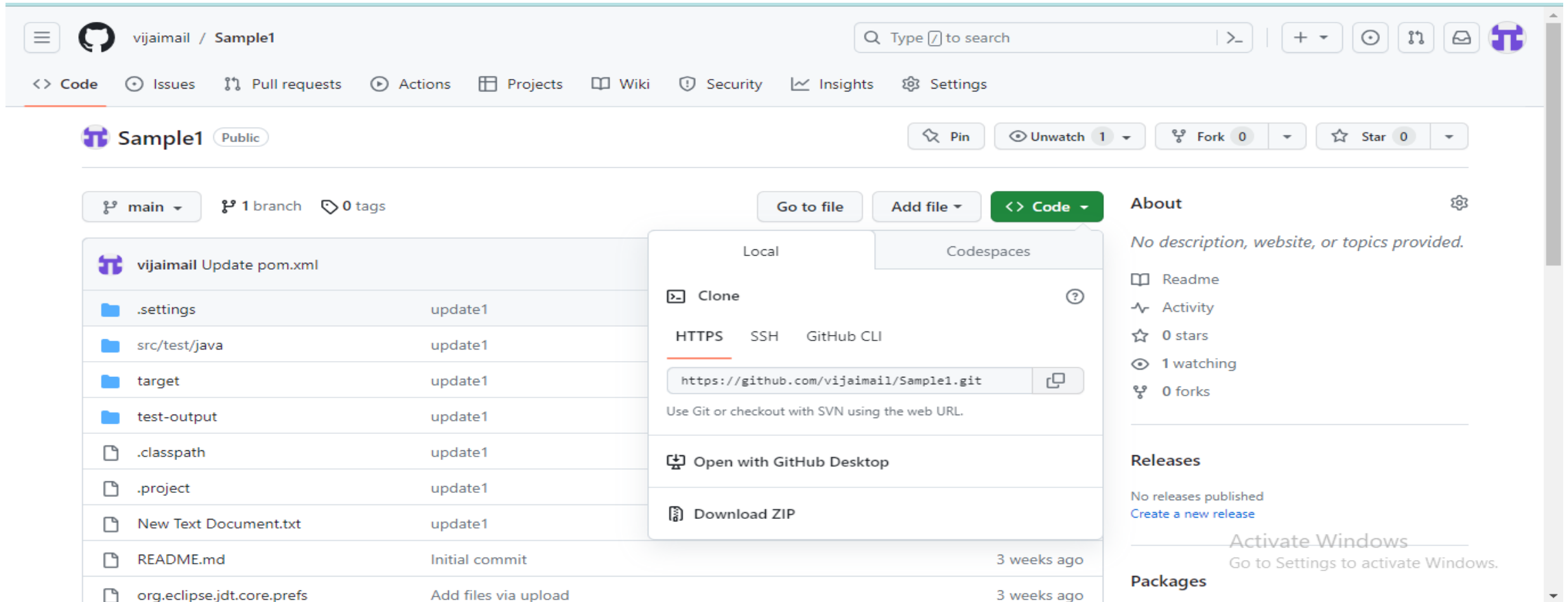
☐ Treat username as secret ?

Password ?

ID ?

Jenkins Job Creation - GitHub

Step 6.3: Github URL of the repository can be found like this.



The screenshot shows the GitHub interface for a repository named 'Sample1' by user 'vijaimail'. The repository is public and has 0 stars, 1 watcher, and 0 forks. The 'Code' dropdown menu is open, displaying the 'Clone' option with the HTTPS URL `https://github.com/vijaimail/Sample1.git`. The repository contains the following files and folders:

File/Folder	Commit	Time
vijaimail Update pom.xml	update1	3 weeks ago
.settings	update1	3 weeks ago
src/test/java	update1	3 weeks ago
target	update1	3 weeks ago
test-output	update1	3 weeks ago
.classpath	update1	3 weeks ago
.project	update1	3 weeks ago
New Text Document.txt	update1	3 weeks ago
README.md	Initial commit	3 weeks ago
org.eclipse.jdt.core.prefs	Add files via upload	3 weeks ago

Jenkins Job Creation - GitHub

Step 6.3: Mention the branch name in repository.

Dashboard > Sample4 > Configuration

Configure

- General
- Source Code Management**
- Build Triggers
- Build Environment
- Pre Steps
- Build
- Post Steps
- Build Settings
- Post-build Actions

Advanced

[Add Repository](#)

Branches to build ?

Branch Specifier (blank for 'any') ? ✖

[Add Branch](#)

Repository browser ?

Additional Behaviours

[Add](#)

[Save](#) [Apply](#)

Activate Windows
Go to Settings to activate Windows.

Jenkins Job Creation - GitHub

Step 6.4: Provide the Root POM name as *pom.xml* and Goals and options as *clean test*.

Dashboard > Sample4 > Configuration

Configure

- General
- Source Code Management
- Build Triggers
- Build Environment
- Pre Steps
- Build**
- Post Steps
- Build Settings
- Post-build Actions

Build

Root POM ?

Goals and options ?

Advanced ▾

Post Steps

☐ Run only if build succeeds

☐ Run only if build succeeds or is unstable

[Save](#) [Apply](#)

Activate Windows

Go to Settings to activate Windows.

Jenkins Job Creation - GitHub

Step 6.5: Add post build steps for Publish TestNG Results, HTML report as per following images

Dashboard > Sample4 > Configuration

☐ E-mail Notification

Configure

- General
- Source Code Management
- Build Triggers
- Build Environment
- Pre Steps
- Build
- Post Steps
- Build Settings**
- Post-build Actions

Post-build Actions

Publish TestNG Results

TestNG XML report pattern ?

Advanced ▾

Add post-build action ▾

Save

Apply

Activate Windows
Go to Settings to activate Windows.
REST API Jenkins 2.401.3

Jenkins Job Creation - GitHub

Step 6.6: Add post build steps for Publish TestNG Results, HTML report as per following images

Dashboard > sample2 > Configuration

Configure

- General
- Source Code Management
- Build Triggers
- Build Environment
- Pre Steps
- Build
- Post Steps
- Build Settings
- Post-build Actions**

Post-build Actions

Publish HTML reports ?

Reports

HTML directory to archive ?

C:\Users\VJ\eclipse-workspace\Sample1\target\surefire-reports

Index page[s] ?

index.html

Index page title[s] (Optional) ?

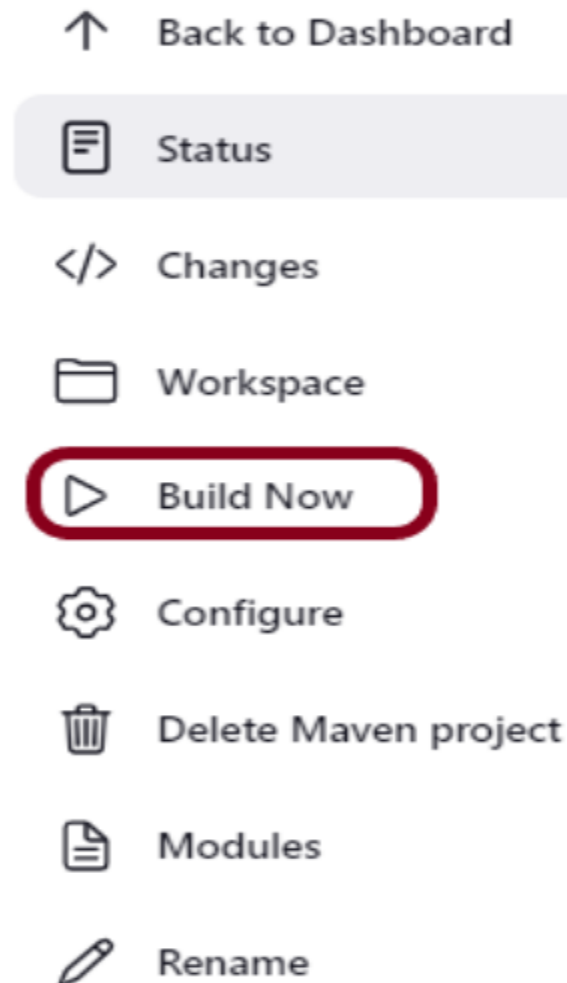
Report title ?

Save

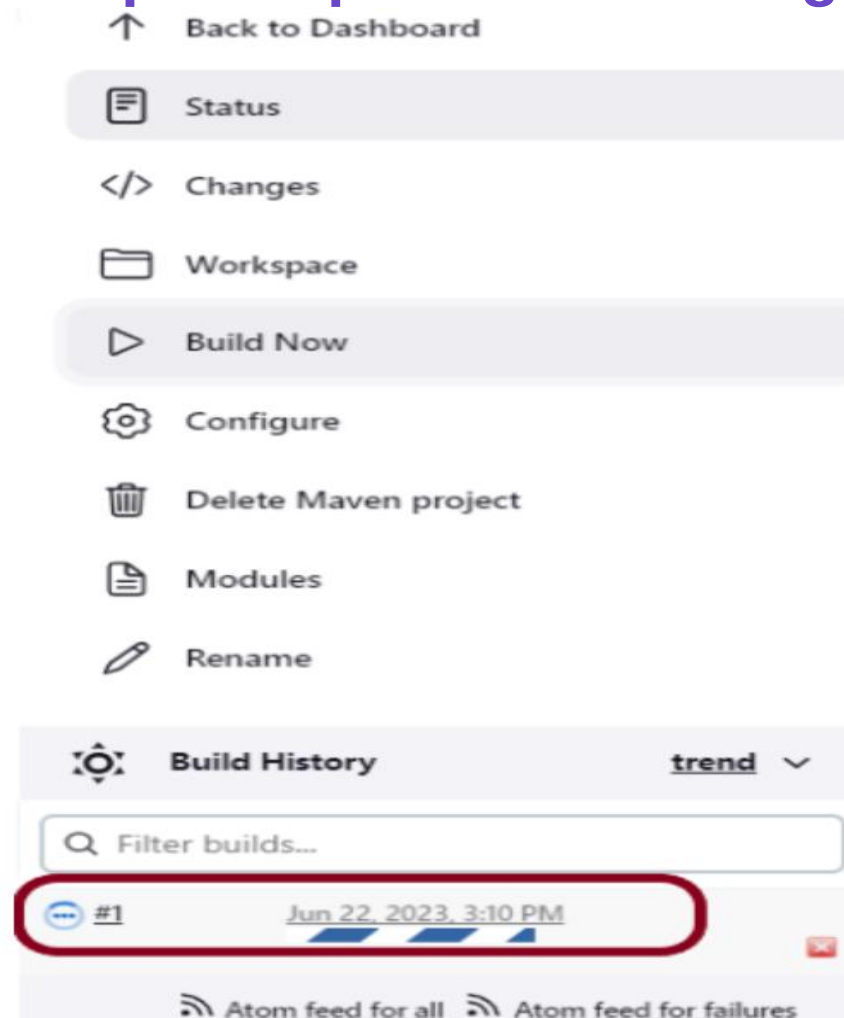
Apply

Activate Windows
Go to Settings to activate Windows.

Step 6.7: Let's execute it now by clicking on the “Build Now” button. You can find the Console Output as per the following



Step 6.7: Let's execute it now by clicking on the “Build Now” button. You can find the Console Output as per the following



Step 6.7: Let's execute it now by clicking on the “Build Now” button. You can find the Console Output as per the following

- ↑ Back to Project
- 📄 Status
- </> Changes
- 📄 Console Output
- 📄 View as plain text
- 📄 Edit Build Information
- 🗑 Delete build '#1'
- 🔗 Git Build Data
- 📄 Test Result
- 🔄 Redeploy Artifacts
- 📊 TestNG Results
- 🔍 See Fingerprints

✓

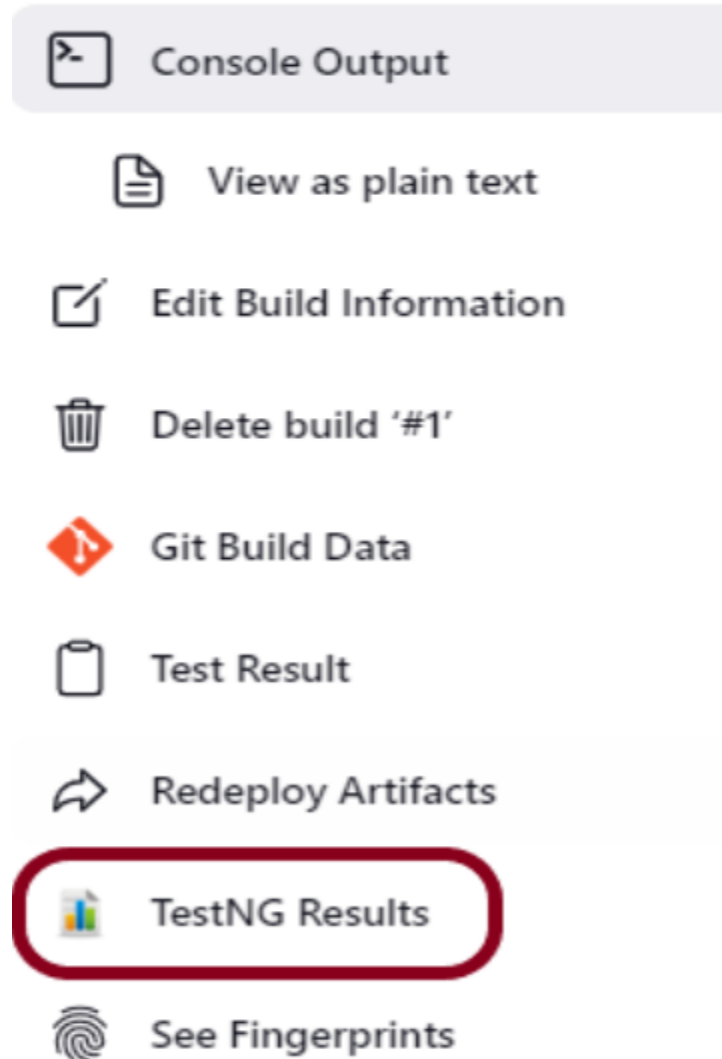
Console Output

```

Started by user unknown or anonymous
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\.jenkins\workspace\GitHub_Demo
The recommended git tool is: NONE
using credential d0cecf22-f23a-4a7a-a9ba-53279474526d
Cloning the remote Git repository
Cloning repository https://github.com/vibssingh/Selenium-TestNG.git
> git.exe init C:\ProgramData\Jenkins\.jenkins\workspace\GitHub_Demo # timeout=10
Fetching upstream changes from https://github.com/vibssingh/Selenium-TestNG.git
> git.exe --version # timeout=10
> git --version # 'git version 2.37.2.windows.2'
using GIT_ASKPASS to set credentials GitHub Credentials
> git.exe fetch --tags --force --progress -- https://github.com/vibssingh/Selenium-TestNG.git +refs/heads/*:ref
> git.exe config remote.origin.url https://github.com/vibssingh/Selenium-TestNG.git # timeout=10
> git.exe config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision e8606a70c295baf2ba1329668cf723783ff0cf05 (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f e8606a70c295baf2ba1329668cf723783ff0cf05 # timeout=10
Commit message: "Added fix for Selenium tests failure on Chrome 111"
First time build. Skipping changelog.
Parsing POMs
Discovered a new module com.example:SeleniumTestNG_Demo SeleniumTestNG_Demo
Modules changed, recalculating dependency graph
Established TCP socket on 60227
[GitHub_Demo] $ java -cp C:\ProgramData\Jenkins\.jenkins\plugins\maven-plugin\WEB-INF\lib\maven35-agent-1.14.jar

```

Step 6.8: View the TestNG Report



Step 6.8: View the TestNG Report

 **Jenkins**

Search (CTRL+K)



 1

 2

Admin ▾

log out

Dashboard > JenkinsMavenGitDemo > #9 > TestNG Results

Status

Changes

Console Output

Edit Build Information

Delete build '#9'

Timings

Git Build Data

Test Result

Redeploy Artifacts

TestNG Results

See Fingerprints

Previous Build

TestNG Results

0 failures(±0)

Failed Tests

No Test method failed

All Tests (grouped by their packages)

hide/expand the table

Package	Duration	Fail	(diff)	Skip	(diff)	Total	(diff)
com.Tests	> 24hrs	0	0	0	0	4	+2

Method 2 - Jenkins Job Creation and executing the code from Selenium Project Directory



Step1: Manage jenkins->Plugins

Dashboard >

+ New Item
 Add description

People
 Build History
 Project Relationship
 Check File Fingerprint
 Manage Jenkins
 My Views

Build Queue
▼

No builds in the queue.

Build Executor Status
▲

built-in node (0 of 2 executors busy)

All

+

S	W	Name ↓	Last Success	Last Failure	Last Duration
		project1	N/A	26 days #2	4 ms
		Sample1	2 days 9 hr #15	20 days #3	1 min 22 sec
		sample2	1 day 8 hr #11	N/A	55 sec
		Sample3	N/A	2 days 10 hr #1	2.1 sec
		Sample4	2 days 8 hr #3	N/A	1 min 3 sec
		Testng Selenium	N/A	2 days 10 hr #4	2.1 sec

Icon:
S
M
L
Icon legend
 Atom feed for all
 Atom feed for failures
 Atom feed for just latest builds

Step1: Manage jenkins->Plugins

Dashboard > Manage Jenkins

System Configuration



System
Configure global settings and paths.



Tools
Configure tools, their locations and automatic installers.

18



Plugins
Add, remove, disable or enable plugins that can extend the functionality of Jenkins.



Nodes and Clouds
Add, remove, control and monitor the various nodes that Jenkins runs jobs on.

Security



Security
Secure Jenkins; define who is allowed to access/use the system.



Credentials
Configure credentials



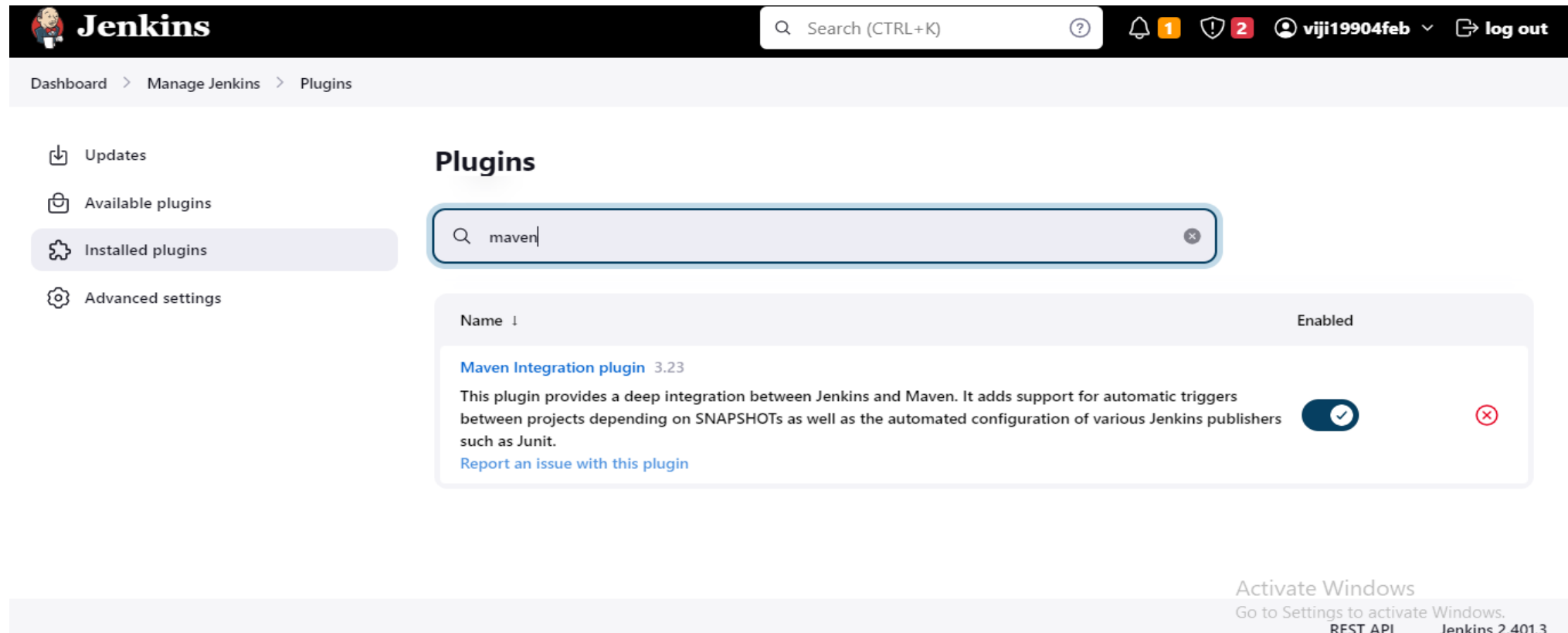
Credential Providers
Configure the credential providers and types

localhost:8082/manage/pluginManager

Activate Windows
Go to Settings to activate Windows.

79 Jenkins | © SmartCliff | Internal | Version 1.0

Step2: Search Maven Integration, testing results and html reports and Install all the three plugins mentioned here.

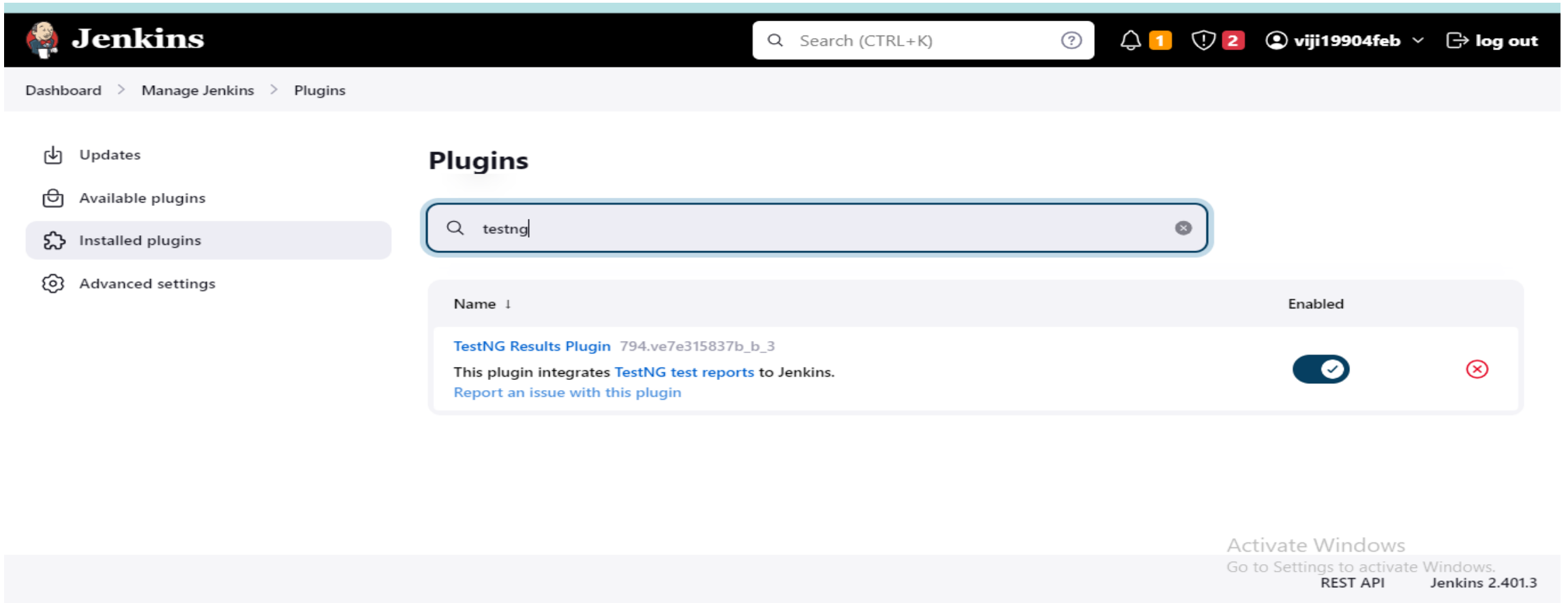


The screenshot shows the Jenkins web interface. At the top is a black header with the Jenkins logo, a search bar, and user information. Below the header is a breadcrumb trail: Dashboard > Manage Jenkins > Plugins. On the left is a sidebar with links to Updates, Available plugins, Installed plugins (which is highlighted), and Advanced settings. The main content area is titled 'Plugins' and features a search bar with 'maven' entered. Below the search bar is a table of installed plugins.

Name ↓	Enabled
Maven Integration plugin 3.23 This plugin provides a deep integration between Jenkins and Maven. It adds support for automatic triggers between projects depending on SNAPSHOTS as well as the automated configuration of various Jenkins publishers such as Junit. Report an issue with this plugin	☒

At the bottom right of the interface, there is a message: 'Activate Windows. Go to Settings to activate Windows.' and a footer bar containing 'REST API' and 'Jenkins 2.401.3'.

Step2: Search Maven Integration, testing results and html reports and Install all the three plugins mentioned here.

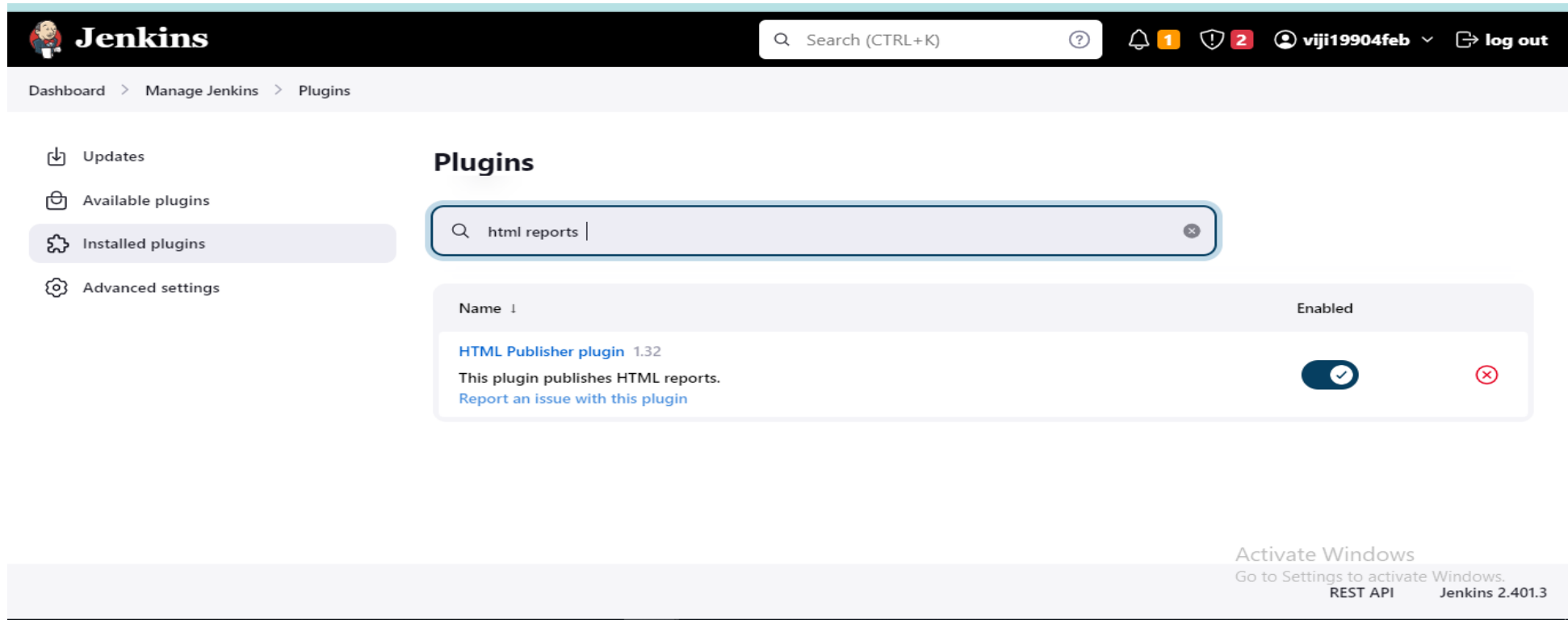


The screenshot shows the Jenkins web interface. At the top is a black header with the Jenkins logo, a search bar, and user information. Below the header is a breadcrumb trail: Dashboard > Manage Jenkins > Plugins. On the left is a sidebar with navigation links: Updates, Available plugins, Installed plugins (highlighted), and Advanced settings. The main content area is titled 'Plugins' and contains a search bar with 'testng' entered. Below the search bar is a table of installed plugins.

Name ↓	Enabled
TestNG Results Plugin 794.ve7e315837b_b_3 This plugin integrates TestNG test reports to Jenkins. Report an issue with this plugin	☒

At the bottom right, there is a 'Activate Windows' watermark and a footer bar with 'REST API' and 'Jenkins 2.401.3'.

Step2: Search Maven Integration, testing results and html reports and Install all the three plugins mentioned here.

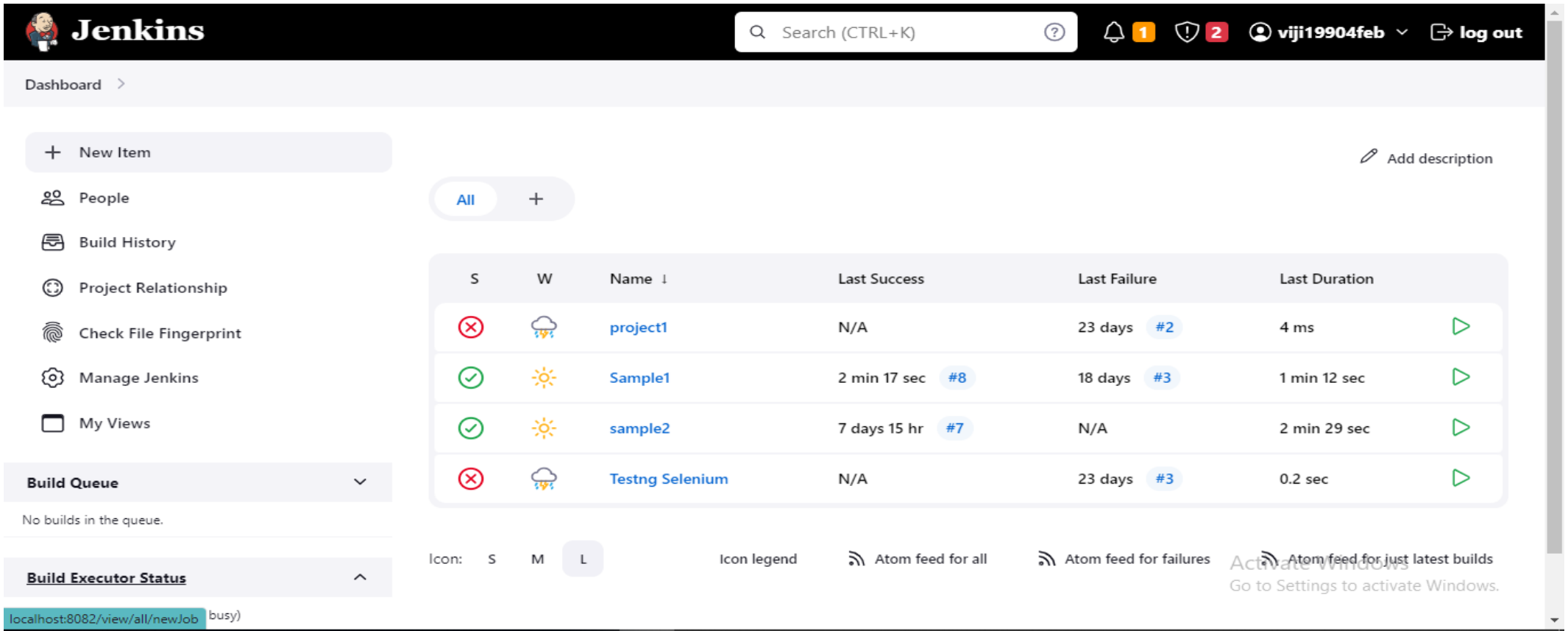


The screenshot shows the Jenkins web interface. At the top is a black header with the Jenkins logo, a search bar, and user information. Below the header is a breadcrumb trail: Dashboard > Manage Jenkins > Plugins. On the left is a sidebar with navigation links: Updates, Available plugins, Installed plugins (highlighted), and Advanced settings. The main content area is titled 'Plugins' and contains a search bar with the text 'html reports'. Below the search bar is a table of installed plugins.

Name ↓	Enabled
HTML Publisher plugin 1.32 This plugin publishes HTML reports. Report an issue with this plugin	☒

At the bottom right of the page, there is a 'Activate Windows' watermark and a footer bar containing 'REST API' and 'Jenkins 2.401.3'.

Step3: After installing the plugins, Click New Item in home page.



The screenshot shows the Jenkins dashboard interface. At the top, there's a navigation bar with the Jenkins logo, a search bar, and user information. Below this is a sidebar with various menu items. The main content area displays a table of build items.

Dashboard >

+ New Item

People

Build History

Project Relationship

Check File Fingerprint

Manage Jenkins

My Views

Build Queue ▾

No builds in the queue.

Build Executor Status ▴

localhost:8082/view/all/newJob (busy)

Table:

S	W	Name ↓	Last Success	Last Failure	Last Duration
❌	☁️	project1	N/A	23 days #2	4 ms
✅	☀️	Sample1	2 min 17 sec #8	18 days #3	1 min 12 sec
✅	☀️	sample2	7 days 15 hr #7	N/A	2 min 29 sec
❌	☁️	Testng Selenium	N/A	23 days #3	0.2 sec

Icon: S M L

Icon legend

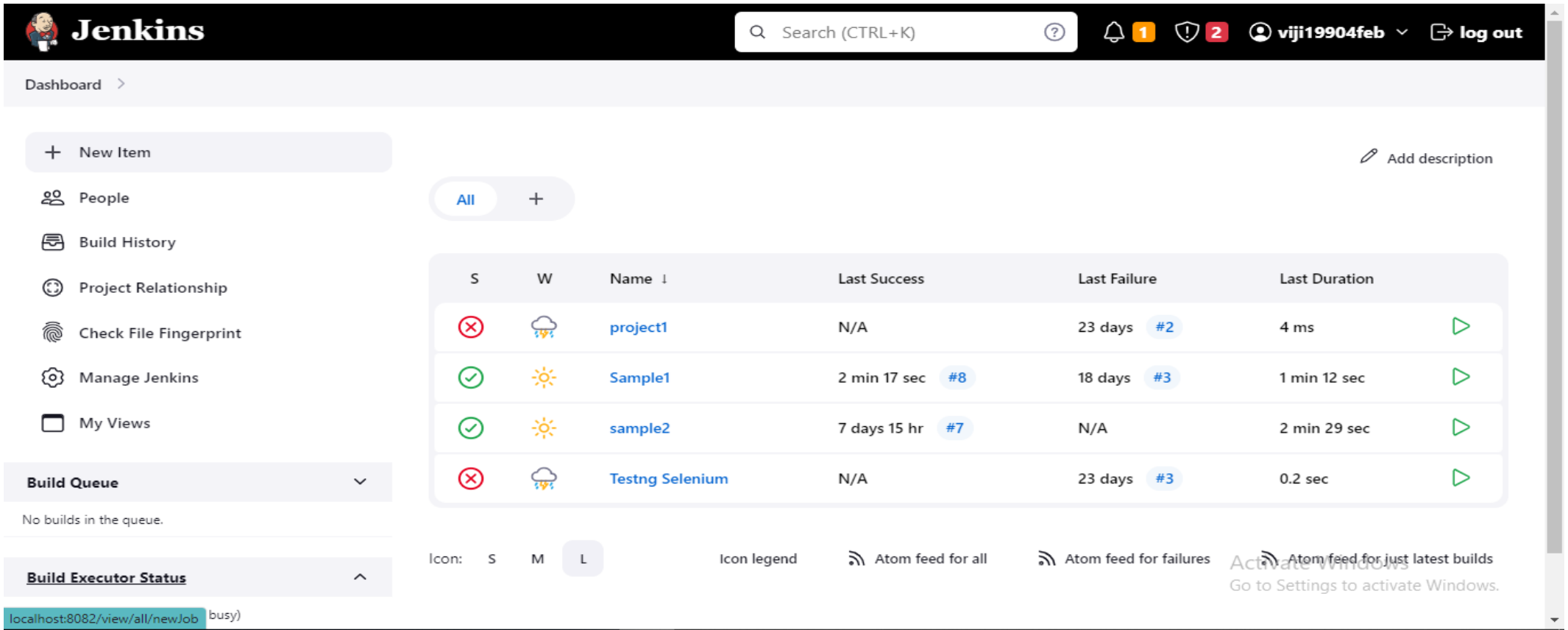
Atom feed for all

Atom feed for failures

Atom feed for just latest builds

Activate Windows
Go to Settings to activate Windows.

Step4: Enter item name and select Maven Project and click OK



The screenshot shows the Jenkins dashboard with the 'All' view selected. The dashboard includes a sidebar with navigation links, a top navigation bar with a search bar and user information, and a main content area displaying a table of build items.

Build Queue: No builds in the queue.

Build Executor Status: localhost:8082/view/all/newJob (busy)

S	W	Name ↓	Last Success	Last Failure	Last Duration
❌	☁️	project1	N/A	23 days #2	4 ms
✅	☀️	Sample1	2 min 17 sec #8	18 days #3	1 min 12 sec
✅	☀️	sample2	7 days 15 hr #7	N/A	2 min 29 sec
❌	☁️	Testng Selenium	N/A	23 days #3	0.2 sec

Icon legend: S (Success), M (Maintenance), L (Loading)

Atom feed for all **Atom feed for failures** **Atom feed for just latest builds**

Go to Settings to activate Windows.

Step4: Enter item name and select Maven Project and click OK


Jenkins






log out

Dashboard > All >

Enter an item name

» Required field


Freestyle project

This is the central feature of Jenkins. Jenkins will build your project, combining any SCM with any build system, and this can be even used for something other than software build.


Maven project

Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.


Pipeline

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.


Multi-configuration project

Activate Windows
Go to Settings to activate Windows.

Step5: It will redirect to Configure Page. In the configure page, In Build Section provide the fullpath of the pom.xml of your project and give Goals as *clean test*

Dashboard > sample2 > Configuration

Configure

- General
- Source Code Management
- Build Triggers
- Build Environment
- Pre Steps
- Build**
- Post Steps
- Build Settings
- Post-build Actions

Build

Root POM ?

Goals and options ?

Advanced ▾

Post Steps

☐ Run only if build succeeds

☐ Run only if build succeeds or is unstable

Save Apply

Activate Windows

Go to Settings to activate Windows.

87 Jenkins | © SmartCliff | Internal | Version 1.0

Step7: Add post build steps for Publish TestNG Results, HTML report as per following images

Dashboard > sample2 > Configuration

Configure

- General
- Source Code Management
- Build Triggers
- Build Environment
- Pre Steps
- Build
- Post Steps
- Build Settings
- Post-build Actions**

Post-build Actions

Publish HTML reports ?

×

Reports

HTML directory to archive ?

C:\Users\VJ\eclipse-workspace\Sample1\target\surefire-reports

Index page[s] ?

index.html

Index page title[s] (Optional) ?

Report title ?

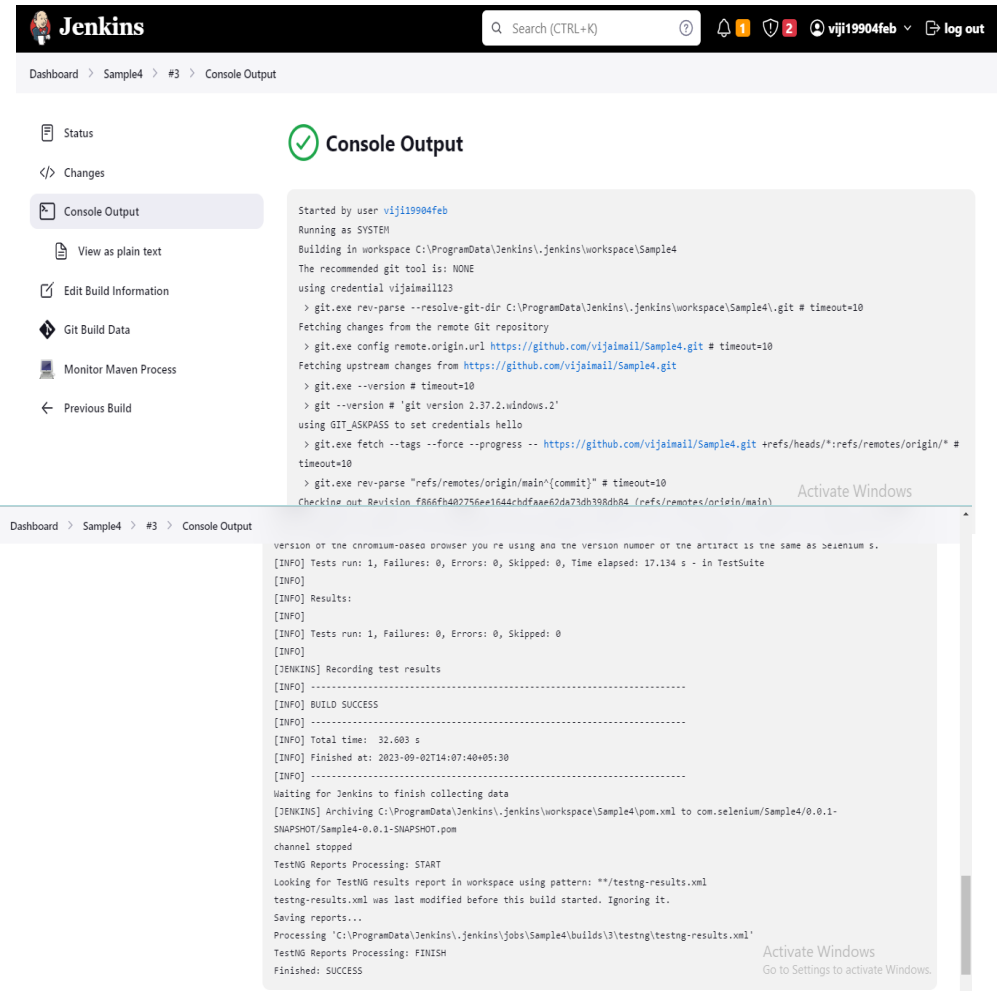
Save

Apply

Activate Windows

Go to Settings to activate Windows.

Step8: Once you click the Build now option ,You can find the Console Output as per the following



Dashboard > Sample4 > #3 > Console Output

Status

<> Changes

Console Output

View as plain text

Edit Build Information

Git Build Data

Monitor Maven Process

Previous Build

Started by user vijji19904feb

Running as SYSTEM

Building in workspace C:\ProgramData\Jenkins\jenkins\workspace\Sample4

The recommended git tool is: NONE

using credential vijjain123

> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\jenkins\workspace\Sample4\.git # timeout=10

Fetching changes from the remote Git repository

> git.exe config remote.origin.url https://github.com/vijjain1/Sample4.git # timeout=10

Fetching upstream changes from https://github.com/vijjain1/Sample4.git

> git.exe --version # timeout=10

> git --version # 'git version 2.37.2.windows.2'

using GIT_ASKPASS to set credentials hello

> git.exe fetch --tags --force --progress -- https://github.com/vijjain1/Sample4.git +refs/heads/*:refs/remotes/origin/* # timeout=10

> git.exe rev-parse "refs/remotes/origin/main^[commit]" # timeout=10

Checking out Revision f866f4b0275f6e1644d4d4f9a662d473d338d8d84 (refs/remotes/origin/main)

version of the chromium-based browser you're using and the version number of the artifact is the same as selenium's.

[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 17.134 s - in TestSuite

[INFO] Results:

[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

[JENKINS] Recording test results

[INFO] BUILD SUCCESS

[INFO] Total time: 32.603 s

[INFO] Finished at: 2023-09-02T14:07:40+05:30

Waiting for Jenkins to finish collecting data

[JENKINS] Archiving C:\ProgramData\Jenkins\jenkins\workspace\Sample4\pom.xml to com.selenium\Sample4\0.0.1-SNAPSHOT.pom

channel stopped

TestNG Reports Processing: START

Looking for TestNG results report in workspace using pattern: **/testng-results.xml

testng-results.xml was last modified before this build started. Ignoring it.

Saving reports...

Processing 'C:\ProgramData\Jenkins\jenkins\jobs\Sample4\builds\3\testng\testng-results.xml'

TestNG Reports Processing: FINISH

Finished: SUCCESS



Dashboard > Sample4 > #3 > Console Output

[INFO] T E S T S

[INFO] -----

[INFO] Running TestSuite

SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".

SLF4J: Defaulting to no-operation (NOP) logger implementation

SLF4J: See <http://www.slf4j.org/codes.html#StaticLoggerBinder> for further details.

ERROR StatusLogger Log4j2 could not find a logging implementation. Please add log4j-core to the classpath. Using SimpleLogger to log to the console...

Starting ChromeDriver 116.0.5845.96 (1a391816688002153ef791ffe60d9e899a71a037-refs/branch-heads/5845@{#1382}) on port 28412

Only local connections are allowed.

Please see <https://chromedriver.chromium.org/security-considerations> for suggestions on keeping ChromeDriver safe.

ChromeDriver was started successfully.

Sep 02, 2023 2:07:29 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch

WARNING: Unable to find CDP implementation matching 116

Sep 02, 2023 2:07:29 PM org.openqa.selenium.chromium.ChromiumDriver lambda\$new\$3

WARNING: Unable to find version of CDP to use for . You may need to include a dependency on a specific version of the CDP using something similar to 'org.seleniumhq.selenium:selenium-devtools-v86:4.8.3' where the version ("v86") matches the version of the chromium-based browser you're using and the version number of the artifact is the same as Selenium's.

[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 17.134 s - in TestSuite

[INFO] Results:

[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

[JENKINS] Recording test results

[INFO] -----

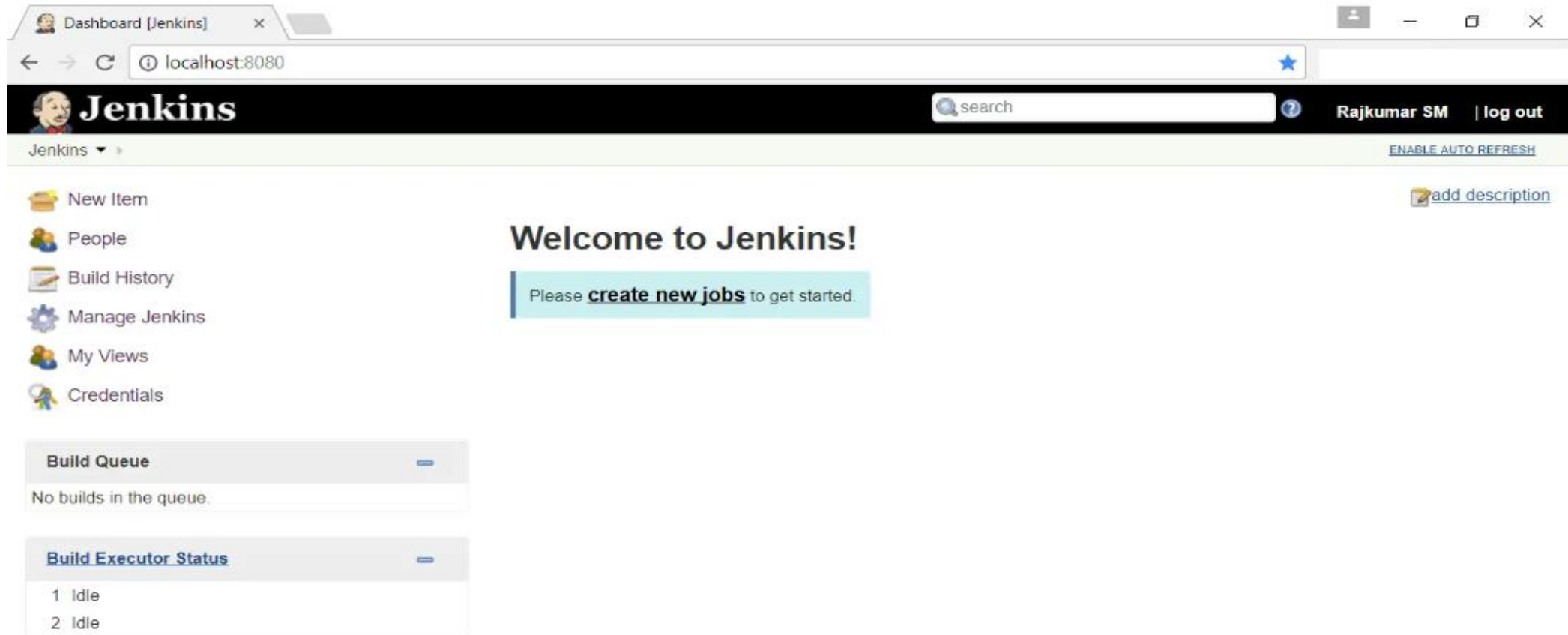
Method 3 - Jenkins Job Creation as a Free Style Project



Jenkins Job Creation

Jenkins Job Creation

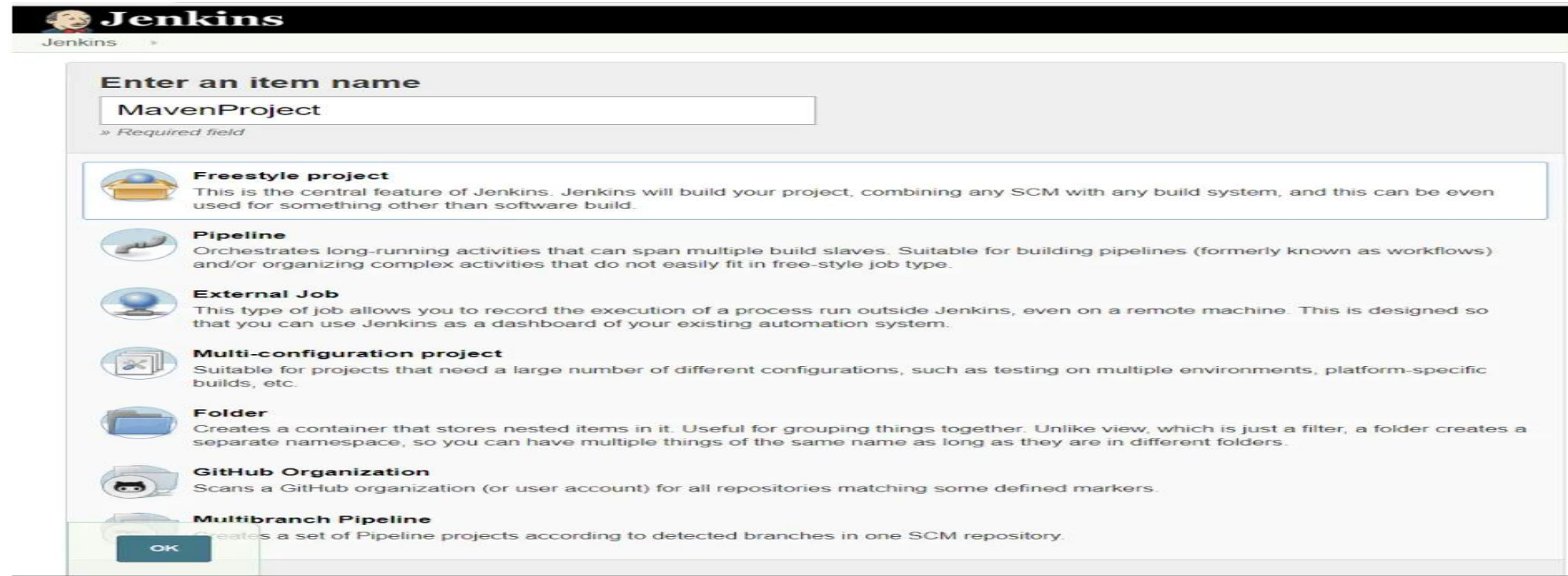
1. Click on **New Item** link to create a job on Jenkins



Jenkins Job Creation

Jenkins Job Creation

2. Enter an item name (here I am adding a name 'MavenProject') and click on FreeStyle Project and click on OK



The screenshot shows the Jenkins 'Create new job' interface. At the top, the Jenkins logo and name are displayed. Below this, the title 'Enter an item name' is followed by a text input field containing 'MavenProject'. A small note below the field indicates it is a required field. The main area lists seven job types, each with an icon and a description:

- Freestyle project**: This is the central feature of Jenkins. Jenkins will build your project, combining any SCM with any build system, and this can be even used for something other than software build.
- Pipeline**: Orchestrates long-running activities that can span multiple build slaves. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
- External Job**: This type of job allows you to record the execution of a process run outside Jenkins, even on a remote machine. This is designed so that you can use Jenkins as a dashboard of your existing automation system.
- Multi-configuration project**: Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder**: Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
- GitHub Organization**: Scans a GitHub organization (or user account) for all repositories matching some defined markers.
- Multibranch Pipeline**: Creates a set of Pipeline projects according to detected branches in one SCM repository.

An 'OK' button is located at the bottom left of the job type selection area.

Jenkins Job Creation

Jenkins Job Creation

3. Scroll down to 'Build' option. Click on 'Add Build Step' and choose the value 'Invoke top-level Maven targets' from the drop down list.

Enter Goals : clean install

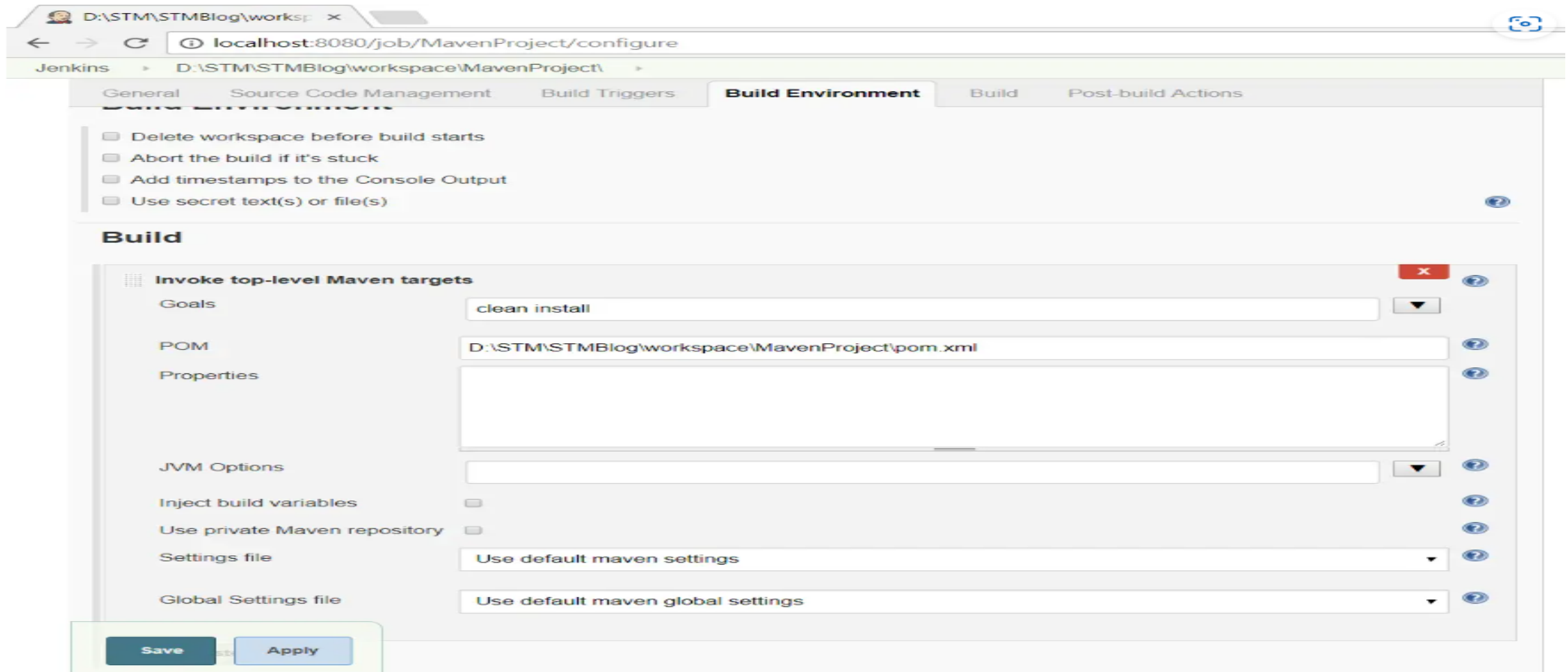
4. Enter POM path (in my case the path is :

D:\STM\STMBlog\workspace\MavenProject\pom.xml)

Click on 'Apply' and 'Save'

Jenkins Job Creation

Jenkins Job Creation

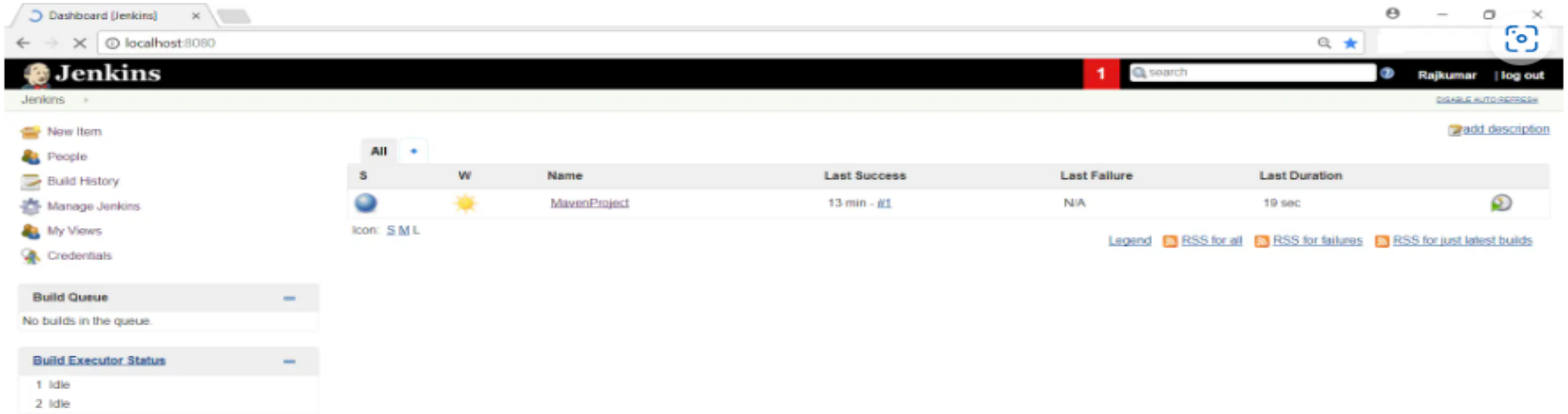


The screenshot shows the Jenkins Job Configuration page for a Maven project. The browser address bar indicates the URL is `localhost:8080/job/MavenProject/configure`. The page has several tabs: General, Source Code Management, Build Triggers, Build Environment, Build, and Post-build Actions. The Build Environment tab is currently selected, showing options like "Delete workspace before build starts", "Abort the build if it's stuck", "Add timestamps to the Console Output", and "Use secret text(s) or file(s)". Below this is the "Build" section, which includes a "Invoke top-level Maven targets" block. This block contains fields for "Goals" (set to "clean install"), "POM" (set to `D:\STM\STMBlog\workspace\MavenProject\pom.xml`), "Properties", "JVM Options", "Inject build variables", "Use private Maven repository", "Settings file" (set to "Use default maven settings"), and "Global Settings file" (set to "Use default maven global settings"). At the bottom of the configuration page are "Save" and "Apply" buttons.

Jenkins Job Creation

Jenkins Job Creation

- Maven Project is created



The screenshot shows the Jenkins Dashboard in a web browser. The browser address bar indicates the URL is localhost:8080. The Jenkins logo and name are at the top left. A search bar and user information (Rajkumar, log out) are at the top right. A red notification badge with the number '1' is visible. On the left sidebar, there are links for 'New Item', 'People', 'Build History', 'Manage Jenkins', 'My Views', and 'Credentials'. The main content area displays a table of jobs. The table has columns for 'S' (Status), 'W' (Icon), 'Name', 'Last Success', 'Last Failure', and 'Last Duration'. One job is listed: 'MavenProject' with a status of 'S' (Success), a sun icon, a last success time of '13 min - #1', and a last duration of '19 sec'. Below the table, there are links for 'icon: S M L' and 'add description'. At the bottom, there are two expandable sections: 'Build Queue' (showing 'No builds in the queue.') and 'Build Executor Status' (showing '1 idle' and '2 idle').

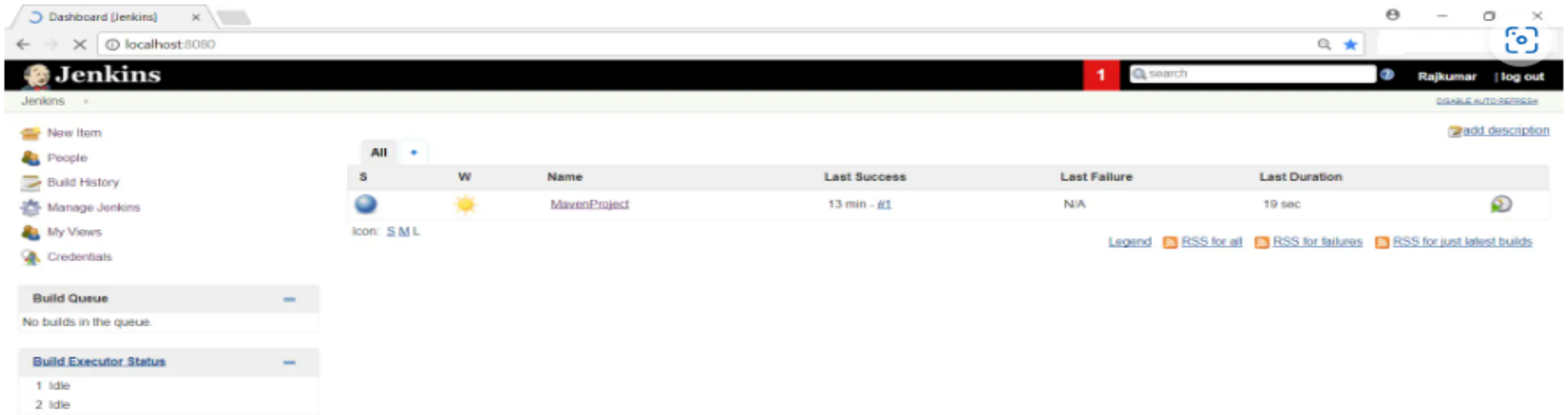
S	W	Name	Last Success	Last Failure	Last Duration
S		MavenProject	13 min - #1	N/A	19 sec

Let's execute it now. Click on '**Build Now**' button. It will invoke pom.xml.

Jenkins Job Creation

Jenkins Job Creation

- Lets execute it. Click on **Build Now** button.



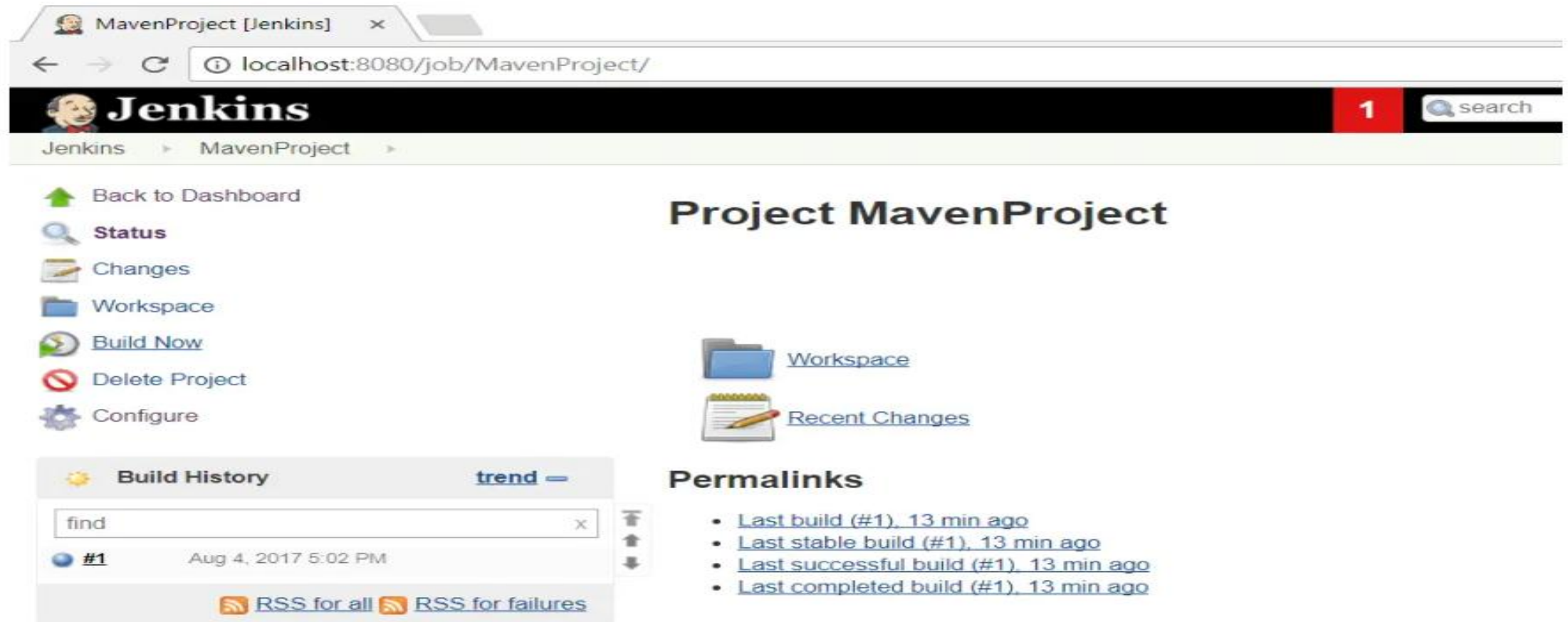
The screenshot shows the Jenkins Dashboard in a web browser. The top navigation bar includes the Jenkins logo, a search bar, and a user profile for 'Rajkumar' with a 'log out' link. A red badge with the number '1' is visible. The left sidebar contains links for 'New Item', 'People', 'Build History', 'Manage Jenkins', 'My Views', and 'Credentials'. The main content area displays a table of jobs. The first job is 'MavenProject', which is in a 'Success' state (indicated by a blue sun icon). The table columns are: S (Status), W (Icon), Name, Last Success, Last Failure, and Last Duration. Below the table, there is a 'Build Queue' section showing 'No builds in the queue.' and a 'Build Executor Status' section showing '1 idle' and '2 idle' executors. At the bottom right of the job entry, there are links for 'Legend', 'RSS for all', 'RSS for failures', and 'RSS for just latest builds'.

S	W	Name	Last Success	Last Failure	Last Duration
Success	Sun	MavenProject	13 min - #1	N/A	19 sec

Let's execute it now. Click on '**Build Now**' button. It will invoke pom.xml.

Jenkins Job Creation

Jenkins Job Creation

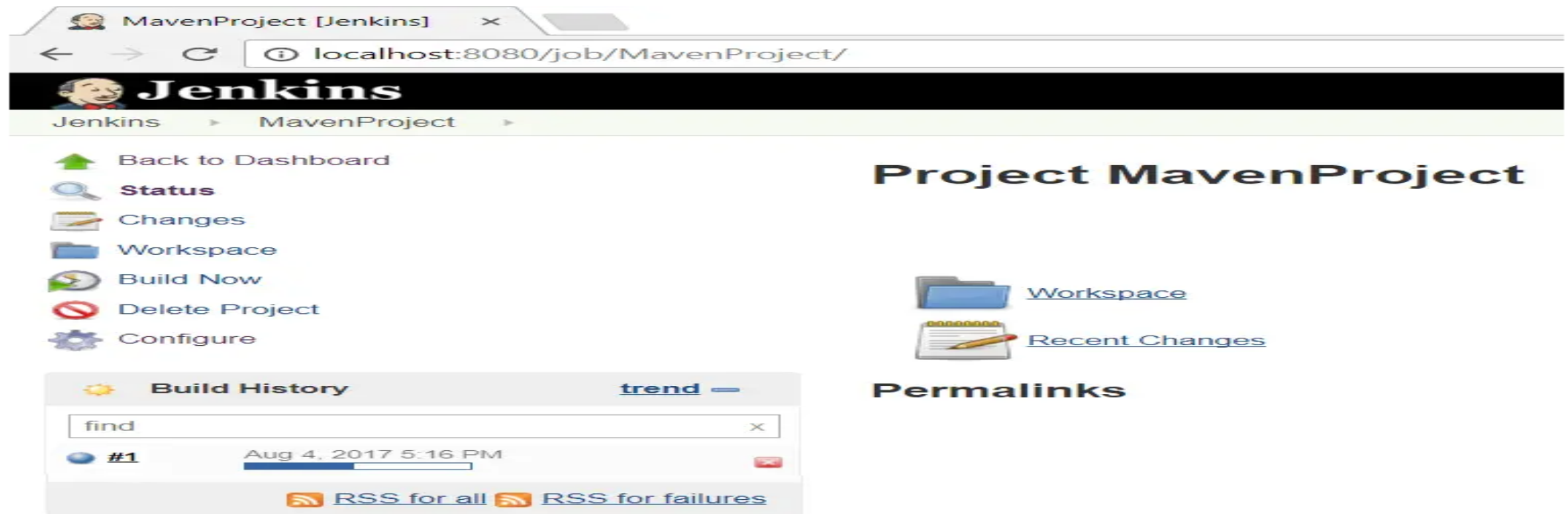


The screenshot shows the Jenkins web interface for a project named 'MavenProject'. The browser address bar shows 'localhost:8080/job/MavenProject/'. The Jenkins logo and name are prominently displayed at the top. A red notification badge with the number '1' is visible in the top right corner. Below the header, there is a search bar and a breadcrumb trail: 'Jenkins > MavenProject >'. On the left sidebar, there are several navigation links: 'Back to Dashboard', 'Status', 'Changes', 'Workspace', 'Build Now', 'Delete Project', and 'Configure'. The main content area is titled 'Project MavenProject'. It contains links to 'Workspace' and 'Recent Changes'. Below this, there is a 'Permalinks' section with four links: 'Last build (#1), 13 min ago', 'Last stable build (#1), 13 min ago', 'Last successful build (#1), 13 min ago', and 'Last completed build (#1), 13 min ago'. At the bottom left, there is a 'Build History' section with a search bar containing the text 'find'. Below the search bar, there is a table with one row showing build '#1' with a timestamp of 'Aug 4, 2017 5:02 PM'. At the bottom of the build history section, there are two RSS feed links: 'RSS for all' and 'RSS for failures'.

Jenkins Job Creation

Jenkins Job Creation

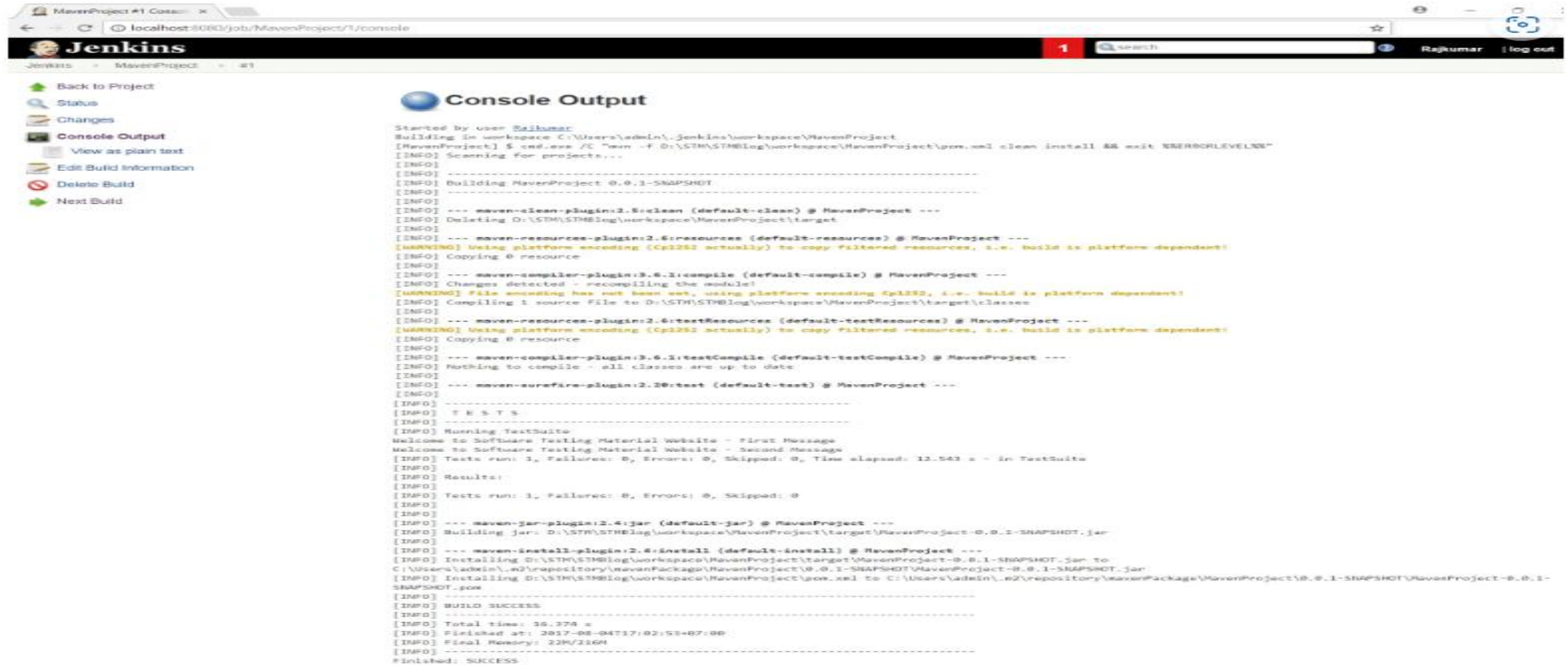
- Right click on Build Number (here in my case it is #1) and click on Console Output to see the result.



You could see Build Status 'Success' on Console Output.

Jenkins Job Creation

Jenkins Job Creation



The screenshot shows the Jenkins web interface with the 'Console Output' tab selected for a job named 'MavenProject #1'. The output log details the build process, including cleaning, resource copying, compilation, testing, and packaging. The build concludes with a 'BUILD SUCCESS' message.

```

Started by user Rajkumar
Building in workspace C:\Users\admin\workspace\MavenProject
[MavenProject] $ cmd.exe /C "mvn -f D:\STM\STMBlog\workspace\MavenProject\pom.xml clean install && exit %ERRORLEVEL%"
[INFO] Scanning for projects...
[INFO]
[INFO] -----
[INFO] Building MavenProject 0.0.1-SNAPSHOT
[INFO] -----
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ MavenProject ---
[INFO] Deleting D:\STM\STMBlog\workspace\MavenProject\target
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MavenProject ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] Copying 0 resource
[INFO]
[INFO] --- maven-compiler-plugin:3.6.1:compile (default-compile) @ MavenProject ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding Cp1252, i.e. build is platform dependent!
[INFO] Compiling 1 source file to D:\STM\STMBlog\workspace\MavenProject\target\classes
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ MavenProject ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] Copying 0 resource
[INFO]
[INFO] --- maven-compiler-plugin:3.6.1:testCompile (default-testCompile) @ MavenProject ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-surefire-plugin:2.20:test (default-test) @ MavenProject ---
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running TestSuite
Welcome to Software Testing Material Website - First Message
Welcome to Software Testing Material Website - Second Message
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 12.543 s - in TestSuite
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ MavenProject ---
[INFO] Building jar: D:\STM\STMBlog\workspace\MavenProject\target\MavenProject-0.0.1-SNAPSHOT.jar
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install) @ MavenProject ---
[INFO] Installing D:\STM\STMBlog\workspace\MavenProject\target\MavenProject-0.0.1-SNAPSHOT.jar to
C:\Users\admin\.m2\repository\mavenPackage\MavenProject\0.0.1-SNAPSHOT\MavenProject-0.0.1-SNAPSHOT.jar
[INFO] Installing D:\STM\STMBlog\workspace\MavenProject\pom.xml to C:\Users\admin\.m2\repository\mavenPackage\MavenProject\0.0.1-SNAPSHOT\MavenProject-0.0.1-SNAPSHOT.pom
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 16.374 s
[INFO] Finished at: 2017-08-04T17:02:53+07:00
[INFO] Final Memory: 22M/216M
[INFO] -----
Finished: SUCCESS
  
```

Scheduling the Build job in Jenkins



Jenkins Job Creation

Jenkins Job Creation

In Configure Page, ***Go to the Build Triggers***

Select the Build periodically option and mention the schedule.

This will open the scheduling text area.

H/5 * * * *

This schedule means that the job will run every 5 minutes.

Dashboard > TestNGReport_Demo >

Configuration

- General
- Source Code Management
- Build Triggers**
- Build Environment
- Pre Steps
- Build
- Post Steps
- Build Settings
- Post-build Actions

☒ Build whenever a SNAPSHOT dependency is built ?
☐ Schedule build when some upstream has no successful builds ?
☐ Trigger builds remotely (e.g., from scripts) ?
☐ Build after other projects are built ?
☒ Build periodically ?

Schedule ?

H/5 * * * *

Would last have run at Thursday, 10 November 2022 at 16:09:34 Greenwich Mean Time;

☐ GitHub hook trigger for GITScm polling ?
☐ Poll SCM ?

Jenkins Job Creation

Jenkins Build run Scheduling

Five fields that are separated by whitespace make up a scheduling item. By adding more than one entry, you can schedule a job for more than one time.

Minute	Hour	Day of Month	Month	Day of week
--------	------	--------------	-------	-------------

Entry	Description	Description
@yearly	Run at any time during the year	H H H H *
@annually	Run at any time during the year	H H H H *
@monthly	Run at any time during the month	H H H * *
@weekly	Run at any time during the week	H H * * H
@daily	Run at any time during the day	H H * * *
@hourly	Run at any time during the hour	H * * * *

Jenkins Job Creation

Jenkins Build run Scheduling

Schedule	Job
* * * * *	Run cron job every minute
H/5 * * * *	Run cron job every 5 minutes
H/30 * * * *	Run cron job every 30 minutes
0 * * * *	Run cron job every hour
0 H/3 * * *	Run cron job every 3 hours
0 13 * * *	Run cron job every day at 1pm
30 2 * * *	Run cron job every day at 2:30 am
0 0 * * *	Run cron job every day at midnight
0 0 * * 0	Run cron job every Sunday
0 0 * * 1	Run cron job every Monday
0 0 1 * *	Run cron job every first day of every month
0 0 1 1 *	Run cron job every first of January every year

THANK YOU